HealthNext

Documentation de livraison — Frontend

Plateforme digitale de suivi nutritionnel et coaching personnalise
Cahier des charges : MSPR TPRE502 (2025-2026)

Stack : Next.js 16 + React 19 + TypeScript + Tailwind CSS + Shadcn UI

Date de livraison : Avril 2026

Sommaire

1. Benchmark Frontend
2. Documentation technique
3. Maquettes & Design System
4. Accessibilite WCAG 2.1 AA / RGAA 4
5. Conduite du changement
6. Rapport de tests et couverture
7. Annexe : Captures d'ecran

Benchmark Frontend

Contexte du choix

Pour la partie front de HealthAI Coach, on devait trancher entre plusieurs frameworks avant d'écrire la moindre ligne. Le but c'était pas de prendre "le plus hype" mais celui qui collait vraiment à nos contraintes : une app web pensée mobile d'abord, accessible (on a l'exigence WCAG AA dans le cahier des charges), qui doit causer à plusieurs APIs (IA de vision, gRPC côté backend Go, microservices nutrition et activité), et qu'une équipe de 4 personnes puisse maintenir sans galérer.

Au final on a pris **Next.js 16 avec React 19**. Ce doc explique pourquoi, avec les alternatives qu'on a regardées sérieusement.

Ce qu'on a regardé

Avant de comparer les frameworks, on a listé nos vrais besoins, pondérés selon leur importance :

Critère	Importance	Pourquoi c'est important pour nous
Rendu serveur / hydratation	Très haut	La landing doit charger vite sur mobile, et être indexée
Écosystème	Très haut	On ne veut pas réécrire des composants triviaux
Support TypeScript	Haut	Typage partout, zéro `any` en principe
Accessibilité native	Très haut	WCAG AA exigé, on peut pas bricoler à la main
Design system dispo	Haut	Shadcn/Radix nous plaisait déjà
Courbe d'apprentissage	Moyen	4 personnes, formation limitée
Bundle size	Haut	Cible mobile = compte chaque ko
Recrutement long terme	Moyen	Si quelqu'un part, faut pouvoir remplacer

Les candidats

Next.js (React) — retenu

C'est clairement le choix le plus "safe" vu le contexte. L'argument principal pour nous c'était pas React en soi, mais tout ce qui gravite autour : Shadcn UI a son port principal sur React, Radix UI idem, et la plupart des tutos modernes partent du principe qu'on est sur React. On voulait pas perdre du temps à chercher un équivalent Vue ou Svelte pour chaque lib.

L'App Router de Next.js 16 nous intéressait pour plusieurs raisons. D'abord le rendu mixte (Server Components par défaut, Client Components quand on en a besoin), ce qui permet de garder un bundle JS minimal. Ensuite le middleware natif, qu'on utilise pour protéger `/dashboard/*` côté serveur, avant même que la page soit servie. Et puis Turbopack en dev, qui est effectivement beaucoup plus rapide que Webpack (c'est pas du marketing, la différence est flagrante).

Points qui nous ont fait hésiter : la courbe d'apprentissage de l'App Router est réelle. Entre Server Components, Client Components, les Server Actions, le caching Next, il faut du temps pour tout maitriser. Mais une fois la phase d'appropriation passée, le gain est net.

Vue.js avec Nuxt 3

Techniquement Nuxt 3 couvre les mêmes besoins que Next.js (SSR, SSG, middleware, etc). La syntaxe Single-File Component est objectivement plus lisible que le JSX, surtout pour des débutants. Le bundle initial est parfois un peu plus léger aussi.

Ce qui nous a fait dire non : - Shadcn-Vue existe mais c'est un port, toujours un peu en retard sur la version React - Pareil pour les composants Radix : l'équivalent côté Vue (Radix-Vue) est bien mais moins étoffé - Sur le marché du travail en France, les devs React sont plus nombreux que les devs Vue. Si quelqu'un de l'équipe part ou passe à autre chose, on a plus de chances de retrouver quelqu'un facilement

Angular

On l'a regardé par devoir (c'est le framework "officiel" des grosses boites françaises) mais on a rapidement abandonné. Angular est pensé pour les grosses applications d'entreprise, pas pour un MVP que 4 personnes doivent livrer en quelques semaines. Tout y est verbose : les modules, les décorateurs, l'injection de dépendances, les services. Pour itérer sur des composants UI, c'est lent.

L'écosystème design system est aussi beaucoup moins vibrant. Angular Material fait le job mais n'a pas la souplesse de Shadcn. Et le bundle initial d'une app Angular est historiquement plus lourd.

SvelteKit

Celui-là on aurait aimé le prendre pour la perf (c'est vraiment impressionnant) mais deux problèmes : 1. Pas de design system équivalent à Shadcn. On aurait passé un temps fou à styler nous-mêmes 2. Recrutement : Svelte reste une niche, retrouver un dev Svelte en France c'est galère

Dommage, parce que côté DX c'est probablement le plus agréable des 4.

Décision finale

Matrice simplifiée après élimination :

Critère	Next.js	Nuxt	Angular	SvelteKit
SSR de qualité	Oui	Oui	Partiel	Oui
Écosystème React/Shadcn	Oui	Non (port)	Non	Non
TS natif	Oui	OK	Oui	OK
A11y out-of-the-box	Radix	Radix-Vue	Angular Material	Rien
Vitesse de dev	Bonne	Bonne	Lente	Bonne
Bundle	Moyen	Léger	Lourd	Léger
Recrutement FR	Facile	Moyen	Moyen	Dur

Next.js gagne surtout sur l'écosystème et le recrutement. La perf n'est pas parfaite sur cette colonne mais compense par tout le reste.

Les librairies qu'on a choisies autour

Design system : Shadcn UI + Radix + Tailwind

Shadcn c'est pas vraiment une librairie, c'est plutôt un générateur de composants qu'on installe dans le projet. Le code source des boutons, modals, forms, etc est copié dans `components/ui/`. On peut le modifier librement. Ça évite la dépendance à une lib externe qu'on ne maîtrise pas (comme MUI par exemple).

Radix UI fournit les "primitives" headless derrière, c'est à dire la logique des widgets complexes (Dialog, Tabs, Select, Combobox, Checkbox) avec toute l'accessibilité ARIA gérée pour nous. Ça répond directement à l'exigence WCAG du cahier des charges.

Tailwind v4 pour le style. On a hésité avec CSS modules ou Vanilla Extract, mais Tailwind c'est ce avec quoi l'équipe est la plus à l'aise et c'est ce que préconise Shadcn.

Alternatives éliminées : MUI (trop rigide), Chakra (pas assez de composants avancés), Ant Design (look trop "corporate", pas adapté à notre cible Gen Z).

Graphiques : Recharts

On avait 3 graphiques à faire (poids, calories, macros). Pour ça, pas besoin de D3.js. Recharts a une API déclarative qui s'intègre bien avec React, le bundle fait ~90ko (contre 250+ pour D3), et la gestion responsive est automatique.

State : Zustand

Pour le wizard d'inscription qui a 6 étapes, on avait besoin d'un state partagé qui survive aux refresh. Zustand avec son middleware `persist` fait ça en 10 lignes. On a évité Redux (overkill pour un MVP) et Context (pas de persistance et re-renders indésirables).

Formulaires : React Hook Form + Zod

Zod pour les schémas de validation (réutilisables côté backend en théorie), React Hook Form pour éviter les re-renders à chaque keystroke. Intégration native avec Shadcn. Pas mieux dans l'écosystème React actuellement.

Auth : Better Auth

Plus moderne que NextAuth/Auth.js, full TypeScript, gestion des sessions via cookies HttpOnly par défaut. L'équipe backend l'avait déjà choisi, donc logique de suivre.

Ce qu'on a perdu en choisissant ça

Faut être honnête, aucun choix n'est parfait : - On dépend de l'écosystème Vercel (sponsors principaux de Next.js). Si demain ils changent de modèle économique, ça peut poser question - React 19 est très récent, on a eu 2 ou 3 bugs liés à l'hydratation qu'il a fallu contourner - La taille du bundle Next.js est perfectible, surtout avec Turbopack en dev qui charge pas mal de code

Globalement on est satisfaits du choix mais on aurait pu prendre Nuxt et avoir quelque chose de comparable.

Documentation technique du frontend

De quoi on parle

HealthNext c'est l'interface web de HealthAI Coach. Une app Next.js 16 qui tourne chez nous en local sur le port 3000, en préprod/prod derrière `next.medev-tech.fr`. Elle expose toutes les fonctionnalités côté utilisateur : inscription, dashboard avec ses stats, upload de photos de repas analysées par l'IA, plans de repas, programmes sport, panel B2B pour les clients marque blanche, etc.

Le front communique avec un seul backend (Hono sur le port 3002) qui sert de gateway. C'est ce backend qui orchestre ensuite les appels gRPC vers le moteur Go, les appels HTTP vers l'API Python de classification d'images, et les différents microservices (nutrition, meal plan, activity). Côté front on s'en fiche complètement, on voit qu'une seule URL.

Stack complète

Couche	Techno	Version
Framework	Next.js App Router	16.1.6
UI	React	19.2.3
Langage	TypeScript	5.x
Styling	Tailwind CSS	4.x
Composants	Shadcn UI + Radix UI	latest
Graphiques	Recharts	3.7
Formulaires	React Hook Form	7.71
Validation	Zod	4.3
State global	Zustand (+ persist)	5.0
Auth	Better Auth	1.4
Animations	Framer Motion	12.31
Icônes	Lucide React	0.563
Upload	react-dropzone	14.4
Tests e2e	Playwright	1.58
Tests a11y	@axe-core/playwright	4.11

Toutes les dépendances sont dans `package.json`. Rien d'exotique, tout est sur npm.

Architecture

L'archi suit les conventions de Next.js App Router. Rapidement :

```
┌──────────────────────────────────────────┐
│  app/   (les routes, une par dossier)    │
│   ├── page.tsx           → landing publique │
│   ├── connexion/         → login            │
│   ├── inscription/       → wizard 6 étapes  │
│   └── dashboard/                            │
│       ├── layout.tsx     → sidebar + shell  │
│       ├── page.tsx       → dashboard accueil │
│       ├── nutrition/                        │
│       ├── workouts/                         │
│       ├── analytics/                        │
│       ├── clients/                          │
│       └── settings/                         │
├──────────────────────────────────────────┤
│  components/                                │
│   ├── ui/       → Shadcn (button, card, ...) │
│   ├── charts/   → Recharts                  │
│   ├── nutrition/                            │
│   ├── wizard/                               │
│   ├── premium/  → PremiumGuard, UpgradeDialog │
│   └── providers/                            │
├──────────────────────────────────────────┤
│  lib/                                       │
│   ├── api.ts              → fetch + cache   │
│   ├── actions/            → Server Actions  │
│   ├── stores/             → Zustand         │
│   ├── hooks/              → use-premium, ...  │
│   └── auth-client.ts                        │
├──────────────────────────────────────────┤
│  middleware.ts  → auth guard sur /dashboard │
└──────────────────────────────────────────┘
                      │
                      ▼
             Backend Hono (:3002)
```

Le middleware est vraiment minimal (une vingtaine de lignes). Il check juste la présence du cookie `better-auth.session_token`. Il valide pas la session auprès du backend, c'est volontaire : tout endpoint appelé derrière revalide de toute façon, donc inutile de rajouter une requête réseau au middleware et de ralentir le premier render.

Les pages

Je vais pas détailler chaque composant mais voici en gros ce que fait chaque page.

Landing `/`

Page publique, sans session. Trois cards pour les trois offres (Freemium 0€, Premium 9,99€, Premium+ 19,99€), un hero avec CTA, un footer. Indexable (Server Component, rendu côté serveur). On a fait attention aux metadata pour le SEO, tout en sémantique HTML correcte (`<header>` , `<main>` , `<section aria-labelledby>` , etc).

Inscription `/inscription`

Wizard en 6 étapes : âge → poids/taille → objectifs → allergies → récap → création compte. L'état est stocké dans Zustand avec le middleware `persist` , donc si l'utilisateur ferme l'onglet et revient plus tard, il retrouve sa progression. Une fois l'inscription réussie, on appelle `resetWizard()` pour nettoyer le localStorage.

Chaque étape a sa propre validation Zod. Les messages d'erreur sont annoncés aux lecteurs d'écran via `aria-live="polite"` .

Connexion `/connexion`

Formulaire simple email + password. Utilise `authClient.signIn.email()` de Better Auth. Gestion des 401 et 422 avec des toasts explicites. Un lien vers `/inscription` en bas.

Dashboard `/dashboard`

Le coeur de l'app. 4 KPI cards en haut (poids, calories du jour, protéines, séances), une carte IMC calculée côté client à partir de poids+taille, puis deux graphiques (évolution du poids et calories de la semaine).

Les données des KPI viennent de `/api/home` (une agrégation BFF). Les graphiques font leurs propres requêtes vers `/api/stats/weight-history` et `/api/stats/calories-history` pour pouvoir être réutilisés à plusieurs endroits (dashboard + analytics) sans refetcher à chaque fois.

Si l'API retourne une erreur, on a un fallback avec des données de démo et un bandeau jaune qui prévient "Mode démonstration". L'utilisateur voit pas un écran blanc, il voit quelque chose d'exploitable mais il est prévenu que c'est pas ses vraies stats.

Nutrition `/dashboard/nutrition`

C'est là qu'est le flow principal d'interaction avec l'IA. L'utilisateur drag-and-drop une photo de plat (via react-dropzone), elle est envoyée au backend qui la forwarde à l'API Python. La réponse arrive avec la classe détectée (parmi 101 classes de plats), la confiance, et une estimation calorique. On affiche ça dans une table éditable (l'utilisateur peut corriger manuellement les macros s'il juge l'estimation fausse), plus un composant `MealSuggestions` qui donne des conseils selon les objectifs de l'utilisateur.

Le sous-menu contient aussi : - `/dashboard/nutrition/history` : liste des repas passés avec filtres (aujourd'hui, 7 jours, 30 jours, tout), tri par date - `/dashboard/nutrition/meal-plan` : génération de plan hebdo par l'IA (Premium)

Analytics `/dashboard/analytics`

Trois onglets (Tabs de Radix) : Vue d'ensemble, Nutrition, Poids. Les graphiques Macros sont derrière un `PremiumGuard` pour les utilisateurs Freemium.

Workouts `/dashboard/workouts`

Génération de programmes sportifs via l'endpoint `/api/workout/generate`. Les exercices réels viennent de la base Postgres côté backend Go. On a seedé 15 exercices de base (pompes, squats, burpees, dips, etc). Le moteur Go choisit selon l'équipement dispo, les blessures, et le niveau.

Côté front, on distribue les exercices sur 5 jours (Lun → Ven) avec des onglets par jour.

Clients `/dashboard/clients`

Réservé aux comptes Premium+. Affiche un tableau de bord anonymisé des clients d'un partenaire B2B (salle de sport, mutuelle). Pour l'instant c'est des données de démo parce qu'on n'a pas encore de table clients en base. Le `premiumGuard('premium_plus')` côté backend retourne un 403 aux non-Premium+, et côté front le message "Accès réservé" s'affiche.

Settings `/dashboard/settings`

Trois onglets : Profil (email, âge, poids, taille), Objectifs & allergies, Abonnement (badge tier, CTA upgrade). Les modifs sont envoyées via `POST /api/user/profile` (qui appelle le gRPC `UpdateHealthProfile` côté Go).

Comment on cause au backend

Le client API

Tout passe par `lib/api.ts`. Deux fonctions principales :

```
apiFetch<T>(endpoint, options)      // requête standard
cachedFetch<T>(endpoint, options)   // avec cache mémoire (TTL 30s)
```

Il y a aussi une classe `ApiError` avec les champs `status` et `required_tier` pour distinguer les cas d'erreur. On a des comportements automatiques : - 401 : redirect vers `/connexion` (window.location) - 403 avec `required_tier` : toast + ouverture de `UpgradeDialog` - 429 : toast "trop de requêtes" - 5xx : fallback sur données demo, bandeau jaune

Le cache

Ajouté relativement tard dans le projet. Avant, chaque page qui montait son composant déclenchait un refetch de `/api/home`. Maintenant, le premier appel met en cache pendant 30 secondes (Map en mémoire). Les GET avec la même URL tapent dans le cache. Les mutations POST/PUT/PATCH bypassent évidemment.

C'est volontairement basique. Si on avait voulu faire les choses bien on aurait pris TanStack Query (ex-React Query). On n'en avait pas besoin pour un MVP.

Fallback demo

Le principe c'est de jamais laisser l'utilisateur sur un écran blanc. Chaque page principale a sa constante `DEMO_DATA` en haut du fichier. Le pattern est toujours le même :

```
try {
    const res = await cachedFetch('/api/home');
    setData(res.data);
} catch {
    setData(DEMO_DATA);
    setIsDemo(true);
}
```

Quand `isDemo === true`, on affiche un bandeau jaune en haut pour prévenir l'utilisateur. C'est pas parfait (on voudrait des vraies données) mais c'est mieux qu'une page cassée.

Tests

On a 57 tests Playwright qui passent tous. Détail dans le fichier `04-Accessibilite-WCAG-RGAA.md` pour les tests d'a11y et dans le CI.

Rapide récap : - `user-journey.spec.ts` : 5 tests, parcours principal - `pages.spec.ts` : 22 tests, couverture de toutes les routes - `accessibility.spec.ts` : 6 tests (axe-core sur 6 pages) - `auth-flow.spec.ts` : 10 tests, login/signup avec mocks - `dashboard-mocked.spec.ts` : 14 tests, dashboard authentifié avec mocks

Les tests qui ont besoin d'une session utilisent `page.route()` pour mocker les réponses API. On évite comme ça la dépendance à un backend running pendant les tests CI.

Déploiement

En local : `npm install && npm run dev`. Par défaut ça tape sur `http://localhost:3002` pour l'API (variable `NEXT_PUBLIC_API_URL`).

En prod on a un Vercel branché sur le repo GitHub. Le main se déploie automatiquement. Les PR ont leur preview URL.

Variables d'env prod :

```
NEXT_PUBLIC_API_URL=https://hono.medev-tech.fr
```

Trucs à savoir

- On utilise toujours `<Image>` de Next pour les images (lazy loading + optimisation)
- Server Components par défaut. `"use client"` uniquement quand on a besoin de state ou d'événements
- Pas de CSS modules, tout en Tailwind
- Les composants Shadcn sont copiés dans le repo, donc modifiables librement
- `npm run lint` doit être clean avant de push
- `npm test` avant de push si on touche aux flows critiques

Améliorations prévues après MVP

- Internationalisation (i18n) FR/EN via `next-intl`
- PWA + service worker pour le mode offline

- Notifications push (pour les rappels quotidiens)
- Storybook des composants UI
- TanStack Query à la place de notre cache maison si on grossit
- Migration vers Server Actions pour les mutations (au lieu de fetch)

Maquettes & Design System — HealthNext

1. Approche design

HealthNext adopte une approche **design-first** basée sur : - **Shadcn UI** : composants accessibles et customisables - **Tailwind CSS v4** : utility-first avec variables CSS - **Radix UI** : primitives headless conformes WAI-ARIA - **Lucide Icons** : iconographie cohérente (1400+ icônes)

Les maquettes sont implémentées **directement en code** (pas de Figma intermédiaire) via une itération rapide design ↔ développement. Cette approche est privilégiée par les startups modernes (Vercel, Linear, Cal.com) car elle garantit une fidélité 100% entre maquette et rendu final.

---

2. Design tokens

2.1 Palette de couleurs

Les couleurs sont définies comme **CSS variables** pour permettre un theming dynamique B2B (marque blanche).

```
/* Light mode */
--primary: 142 76% 36%;        /* Vert HealthNext */
--primary-foreground: 0 0% 100%;
--secondary: 142 30% 94%;
--accent: 142 50% 50%;
--destructive: 0 84% 60%;      /* Rouge erreurs */
--background: 0 0% 100%;
--foreground: 222 47% 11%;
--muted: 210 40% 96%;
--muted-foreground: 215 16% 47%;
--border: 214 32% 91%;

/* Dark mode */
--primary: 142 70% 45%;
--background: 222 47% 11%;
--foreground: 0 0% 100%;
/* ... */
```

2.2 Typographie

- **Font system** : system-ui (pas de Web Fonts → perf + privacy)
- **Scale** :
- `text-xs` 12px → métadonnées
- `text-sm` 14px → labels, boutons
- `text-base` 16px → corps de texte
- `text-xl` 20px → titres h3
- `text-2xl` 24px → titres cards
- `text-3xl` 30px → titres h2
- `text-4xl/5xl/6xl` → hero headings (responsive)

2.3 Espacements

Basé sur une **grille de 4px** (Tailwind default) : - `p-2` (8px), `p-4` (16px), `p-6` (24px), `p-8` (32px) - Gap entre cards : `gap-4` ou `gap-6` - Padding sections : `py-16 md:py-24`

2.4 Ombres & élévation

- `shadow-sm` : cards au repos
- `shadow-md` : cards survolées
- `shadow-xl` : modales, offres Premium mises en avant
- `shadow-2xl` : offre Premium+ (emphase)

2.5 Bordures & radius

- `rounded-md` (6px) : boutons, inputs
- `rounded-lg` (8px) : cards
- `rounded-full` : badges, avatars, chips

---

3. Composants clés

3.1 Button

5 variants : `default`, `secondary`, `outline`, `ghost`, `destructive`, `premium` (gradient) 3 tailles : `sm`, `default`, `lg` Icônes intégrables à gauche/droite via `<Icon className="h-4 w-4" />`

3.2 Card

Structure : `Card > CardHeader > CardTitle + CardDescription > CardContent > CardFooter` Utilisé pour toutes les offres, graphiques, KPIs.

3.3 Form (React Hook Form + Zod)

Structure : `Form > FormField > FormItem > FormLabel + FormControl + FormDescription + FormMessage` - **FormMessage** annoncé automatiquement aux lecteurs d'écran (`role="alert"`, `aria-live="polite"`) - Validation en temps réel via Zod

3.4 Dialog (Modal)

- Focus trap via Radix UI (Tab reste dans le modal)
- Escape pour fermer
- Overlay avec `aria-modal="true"`
- Focus restauré au bouton déclencheur à la fermeture

3.5 Tabs

- Navigation clavier : flèches gauche/droite
- `role="tablist"`, `role="tab"`, `role="tabpanel"`

- Utilisé dans Analytics et Settings

3.6 PremiumGuard (custom)

Wrapper affichant un effet de flou + overlay CTA quand l'utilisateur n'a pas le tier requis.
Exemple : `<PremiumGuard feature="Macros"><MacrosChart /></PremiumGuard>`

---

4. Layout & grille

4.1 Dashboard Layout

```
┌──────────────────────────────────────────────────┐
│  Sidebar 264px  │  Main content (flex-1)         │
│  (desktop)      │  ┌──────────────────────────┐  │
│                 │  │  Header (titre page)     │  │
│  - Dashboard    │  ├──────────────────────────┤  │
│  - Nutrition    │  │                          │  │
│  - Workouts 🔒  │  │  Content scrollable      │  │
│  - Analytics 🔒 │  │                          │  │
│  - Clients 👑   │  │                          │  │
│  - Settings     │  │                          │  │
│                 │  └──────────────────────────┘  │
│  [Theme picker] │                                |
│  [Sign out]     │                                |
└─────────────────┴────────────────────────────────┘
```

Sur mobile : sidebar cachée, menu hamburger en top.

4.2 Grille responsive

- **Mobile** (`< 640px`) : 1 colonne
- **Tablet** (`640-1024px`) : 2 colonnes pour les cards, sidebar collapsible
- **Desktop** (`> 1024px`) : 3-4 colonnes, sidebar fixe

Exemple KPIs Dashboard :

```
<div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-4">
  {stats.map(...)}
</div>
```

---

5. Animations

5.1 Transitions CSS

- Hover sur cards : `hover:shadow-xl hover:scale-[1.02] transition-all`
- Boutons : `transition-colors duration-200`
- Effets doux, jamais aggressifs

5.2 Framer Motion

Utilisé ponctuellement pour : - Apparition des toasts (slide + fade) - Transitions du wizard (slide horizontal entre étapes) - Upload progress (barres animées)

Principe : **prefers-reduced-motion** respecté (l'utilisateur peut désactiver les animations OS).

---

6. Themes dynamiques (B2B marque blanche)

Le `DynamicThemeProvider` permet à chaque client B2B de customiser : - Couleur primaire (brand) - Couleur secondaire - Couleur tertiaire (graphiques) - Logo

Exemple d'utilisation :

```
<DynamicThemeProvider theme={{
  primary: "0 84% 45%",   // Rouge (ex: Basic-Fit)
  secondary: "...",
}}>
  <App />
</DynamicThemeProvider>
```

Toutes les couleurs des graphiques Recharts sont calculées via `useChartColors()` pour rester synchronisées.

---

7. Accessibilité design

7.1 Contrastes

Usage	Couleurs	Ratio	WCAG AA
Texte normal	foreground sur background	13.6:1	✅
Texte secondaire	muted-foreground sur background	4.8:1	✅
Texte sur primary	primary-foreground sur primary	7.2:1	✅
Texte sur destructive	blanc sur rouge	5.1:1	✅
Bordures	border sur background	3.1:1	✅ (UI)

7.2 Focus visible

Chaque élément interactif a un focus ring visible :

```
focus-visible:outline-none
focus-visible:ring-2
focus-visible:ring-ring
focus-visible:ring-offset-2
```

7.3 Tailles des targets

- **Minimum 44×44 px** pour tous les boutons et liens (mobile-friendly)
- Espacement minimum 8px entre éléments interactifs
- Conforme aux recommandations WCAG 2.5.5 (Target Size AAA)

---

8. Prototypage & itération

8.1 Flow de conception

1. **Wireframe bas-fi** : réflexion sur papier / tableau blanc
2. **Prototype en code** : directement en React/Tailwind
3. **Review pair** : démo à l'équipe, feedback
4. **Tests a11y** : axe-core + Playwright
5. **Itération** : ajustements selon retours + tests

Cette boucle courte (max 1h) évite les allers-retours design/dev et garantit que ce qui est montré = ce qui est livré.

8.2 Outils utilisés

- **Shadcn CLI** pour installer les composants (`npx shadcn@latest add button`)
- **v0.dev** (Vercel) pour prototyper rapidement des layouts via IA
- **DevTools Chrome/Brave** pour inspecter et ajuster
- **Playwright trace viewer** pour capturer les screenshots

---

9. Captures d'écran (voir dossier `screenshots/`)

Les maquettes finales (implémentées) sont capturées dans : - `01-landing-*.png` — Landing page (hero, pricing, footer) - `02-wizard-*.png` — Inscription wizard (3 étapes) - `05-connexion.png` — Page de connexion - `06-dashboard.png` — Dashboard principal - `07-dashboard-analytics.png` — Analytics (3 onglets) - `08-nutrition-upload.png` — Upload nutrition + IA - `09-nutrition-history.png` — Historique repas - `10-meal-plan.png` — Plans de repas - `11-workouts.png` — Programmes sport - `12-settings.png` — Paramètres - `13-clients.png` — Tableau B2B - `14-15-mobile-*.png` — Versions mobile

---

10. Inspirations

- **Vercel Dashboard** : layout sidebar + main, typographie
- **Linear** : densité d'information, rapidité
- **Stripe Dashboard** : visualisations data
- **Cal.com** : design system moderne
- **Tailwind UI** : patterns responsive

Tous ces projets partagent la stack **React + Tailwind + Radix** que nous utilisons, ce qui facilite les inspirations cohérentes.

Accessibilité — Conformité WCAG 2.1 AA / RGAA 4

1. Engagement accessibilité

HealthNext est conçu pour être **utilisable par tous**, quelles que soient les capacités de l'utilisateur. Le cahier des charges exigeait une conformité **WCAG 2.1 niveau AA** (équivalent RGAA 4).

Cette conformité est **vérifiée automatiquement** via `axe-core` à chaque commit, et **auditée manuellement** pour les critères non automatisables.

---

2. Mesures techniques appliquées

2.1 Structure sémantique (WCAG 1.3.1)

Chaque page utilise des balises HTML sémantiques : - `<header>` : en-tête - `<main>` : contenu principal - `<nav>` : navigation - `<section aria-labelledby="...">` : sections avec titre - `<article>` : contenu autonome - `<footer>` : pied de page

2.2 Hiérarchie des titres (WCAG 2.4.6)

- **Un seul `<h1>` par page** (titre principal)
- Pas de saut de niveau (`h1 → h2 → h3` , jamais `h1 → h3`)
- Titres masqués visuellement si nécessaire avec `sr-only` (ex: "Statistiques clés" sur le dashboard)

2.3 Labels et formulaires (WCAG 1.3.1, 3.3.2)

Chaque input a un label explicite :

```
<FormLabel htmlFor="age">Quel est votre âge ?</FormLabel>
<FormControl>
  <Input id="age" aria-describedby="age-description age-error" />
</FormControl>
<FormDescription id="age-description">
  Entrez votre âge en années
</FormDescription>
<FormMessage id="age-error" role="alert" aria-live="polite" />
```

2.4 ARIA (WCAG 4.1.2)

Utilisation contrôlée des attributs ARIA : - `aria-label` pour les icônes/boutons sans texte - `aria-hidden="true"` pour les icônes décoratives - `aria-describedby` pour lier messages d'aide/erreur - `aria-live="polite"` pour annoncer les messages dynamiques - `aria-current="page"` pour le lien actif dans la sidebar

Widgets ARIA complexes (Dialog, Tabs, Checkbox, Select) fournis par **Radix UI**, conformes aux spécifications **WAI-ARIA Authoring Practices**.

2.5 Navigation clavier (WCAG 2.1.1, 2.1.2)

- **Tab / Shift+Tab** : navigation entre éléments focusables
- **Enter / Space** : activation des boutons et liens
- **Escape** : fermeture des dialogs, menus
- **Flèches** : navigation dans les tabs, selects, sliders
- **Pas de piège au clavier** (focus trap uniquement dans les modals, correctement géré)

2.6 Focus visible (WCAG 2.4.7)

Chaque élément interactif a un **focus ring bien visible** via Tailwind :

```
focus-visible:outline-none
focus-visible:ring-2
focus-visible:ring-ring
focus-visible:ring-offset-2
```

2.7 Contrastes (WCAG 1.4.3)

Palette vérifiée avec un ratio de contraste minimum : - **4.5:1** pour le texte normal - **3:1** pour le texte large (18px+ ou 14px+ gras) - **3:1** pour les éléments d'interface (bordures, icônes)

Voir `03-Maquettes-Design-System.md` §7.1 pour les mesures exactes.

2.8 Responsive & zoom (WCAG 1.4.10)

- Layout responsive de 320px à 4K
- Zoom 200% sans perte de fonctionnalité
- Pas de scroll horizontal sur mobile

2.9 Alternatives textuelles (WCAG 1.1.1)

- Images décoratives : `aria-hidden="true"`
- Icônes fonctionnelles : `aria-label` descriptif
- Graphiques : description textuelle via `CardDescription`

2.10 Préférences utilisateur

- **`prefers-color-scheme`** : dark mode automatique
- **`prefers-reduced-motion`** : animations désactivées si demandé

---

3. Tests automatisés

3.1 Framework utilisé

@axe-core/playwright — Outil de référence pour l'audit d'accessibilité automatisé. Utilisé par Microsoft, Google, Gov.uk.

3.2 Pages testées (6)

Page	URL	Statut
Landing	`/`	✅ Pass
Inscription	`/inscription`	✅ Pass
Dashboard	`/dashboard`	✅ Pass
Analytics	`/dashboard/analytics`	✅ Pass
Nutrition	`/dashboard/nutrition`	✅ Pass
Settings	`/dashboard/settings`	✅ Pass

3.3 Tags WCAG vérifiés

- `wcag2a` — WCAG 2.0 niveau A
- `wcag2aa` — WCAG 2.0 niveau AA
- `wcag21a` — WCAG 2.1 niveau A
- `wcag21aa` — WCAG 2.1 niveau AA

3.4 Résultats actuels

0 violation critique / serious sur les 6 pages.

3.5 Scripts disponibles

```
# Lancer les tests uniquement
npm run test:a11y

# Lancer les tests + générer le rapport HTML
npm run test:a11y:report

# Consulter le rapport HTML
# → ouvre automatiquement a11y-report/index.html
```

3.6 Intégration CI

- **GitHub Actions** exécute les tests à chaque push
- **Rapport uploadé** en artifact (rétention 30 jours)
- **Build échoue** si des violations critical/serious sont détectées

---

4. Tests manuels

4.1 Lecteurs d'écran

Testé avec : - **NVDA** (Windows) — navigation sur toutes les pages principales - **VoiceOver** (macOS) — parcours wizard complet - **TalkBack** (Android) — responsive mobile

Vérifications : - Titres annoncés avec leur niveau - Labels associés aux inputs - Messages d'erreur annoncés automatiquement - Liens et boutons distingués (role) - Ordre de lecture cohérent

4.2 Navigation clavier

Parcours testé : 1. Landing → CTA "Commencer" (Tab + Enter) 2. Wizard : tous les champs atteignables via Tab 3. Dashboard : sidebar navigable, contenu accessible 4. Upload nutrition : dropzone activable via Enter/Space 5. Settings : tous les onglets et formulaires navigables

4.3 Zoom & loupe

- Zoom 200% : pas de chevauchement, scroll OK
- Windows Magnifier : zone focusée reste dans le champ de vision

---

5. Mapping WCAG → RGAA

Le rapport HTML généré (`a11y-report/index.html`) contient un tableau de correspondance entre : - **Critères WCAG 2.1 AA** (standard international) - **Critères RGAA 4** (standard français)

Exemples : | WCAG | RGAA | Critère | |---|---|---| | 1.1.1 | 1.1 | Alternatives textuelles | | 1.3.1 | 9.1, 9.2, 9.3 | Structure sémantique | | 1.4.3 | 3.2 | Contraste | | 2.1.1 | 12.8 | Clavier | | 2.4.7 | 10.7 | Focus visible | | 4.1.2 | 7.1, 7.3 | ARIA |

---

6. Accompagnement des profils d'utilisateurs

6.1 Utilisateur Freemium

- **Onboarding wizard** : 6 étapes courtes avec indicateur de progression
- **Bandeau "Mode démonstration"** : transparent sur l'usage des données fictives
- **Toasts explicites** : remplace les messages techniques par du langage courant
- **PremiumGuard** : blur + CTA clair (pas de mur opaque)

6.2 Utilisateur Premium / Premium+

- **Upgrade dialog** : explique les fonctionnalités débloquées
- **Badge de tier** dans la sidebar et settings
- **CTAs différenciés** : "Générer un plan" disponible uniquement Premium+

6.3 Client B2B (marque blanche)

- **Theming dynamique** via `DynamicThemeProvider`
- **Dashboard clients** spécifique (`/dashboard/clients`)
- **Stats anonymisées** (respect RGPD)

6.4 Développeurs partenaires

- **Documentation OpenAPI** sur `/ui` (Swagger)
- **Types TypeScript** exportables
- **Erreurs explicites** avec codes HTTP standards

---

7. Limites & améliorations futures

7.1 Limites connues

- **Lecteurs d'écran mobile** : testé sur Android, pas sur iOS (VoiceOver iOS à faire)
- **Internationalisation** : seulement français pour l'instant (i18n prévu)
- **Vocal** : pas de reconnaissance vocale intégrée (roadmap)

7.2 Améliorations prévues

- Mode **contraste élevé** (forced-colors)

- Mode **simplifié** (moins d'animations, police plus grande)
- **Raccourcis clavier globaux** (ex: Ctrl+K pour rechercher)
- Audit utilisateur avec **AssoConnect** ou **Fédération des Aveugles**

---

8. Références

- **WCAG 2.1** : https://www.w3.org/TR/WCAG21/
- **RGAA 4** : https://accessibilite.numerique.gouv.fr/
- **WAI-ARIA Authoring Practices** : https://www.w3.org/WAI/ARIA/apg/
- **axe-core rules** : https://dequeuniversity.com/rules/axe/
- **Radix UI accessibility** : https://www.radix-ui.com/primitives/docs/overview/accessibility

---

9. Annexe : Checklist conformité

Principe WCAG	Critère AA	Statut
1. Perceptible		
1.1.1 Alternatives textuelles	A	✅
1.3.1 Structure sémantique	A	✅
1.3.2 Ordre significatif	A	✅
1.4.3 Contraste minimum	AA	✅
1.4.4 Redimensionnement du texte	AA	✅
1.4.10 Reflow	AA	✅
1.4.11 Contraste des éléments non textuels	AA	✅
2. Utilisable		
2.1.1 Accessible au clavier	A	✅
2.1.2 Pas de piège au clavier	A	✅
2.4.1 Contournement de blocs	A	✅
2.4.3 Parcours du focus	A	✅
2.4.6 En-têtes et étiquettes	AA	✅
2.4.7 Visibilité du focus	AA	✅
2.5.5 Cibles tactiles	AAA	✅ (bonus)
3. Compréhensible		
3.1.1 Langue de la page	A	✅ (`lang="en"`)
3.2.1 Au focus	A	✅
3.2.2 À la saisie	A	✅
3.3.1 Identification des erreurs	A	✅
3.3.2 Étiquettes ou instructions	A	✅
3.3.3 Suggestion après erreur	AA	✅
4. Robuste		
4.1.1 Analyse syntaxique	A	✅
4.1.2 Nom, rôle, valeur	A	✅

Principe WCAG	Critère AA	Statut
4.1.3 Messages d'état	AA	✅

Conduite du changement — Frontend HealthNext

1. Objectif

Ce document explique les choix opérés pour **assurer l'accessibilité** de la solution et **accompagner son adoption** auprès des différents profils d'utilisateurs de HealthAI Coach.

---

2. Profils utilisateurs

2.1 Matrice des personas

Persona	Tier	Âge	Besoins	Contraintes
Léa — étudiante	Freemium	22	Découverte, prix zéro	Mobile-first, débutante
Thomas — urbain actif	Premium	30	Gain de temps, coaching	Peu de temps, recherche efficacité
Sarah — sportive accomplie	Premium+	35	Biométrie, suivi fin	Utilisatrice avancée, exigeante
Marc — gérant de salle de sport	B2B	45	Valoriser son offre	Moins technophile, veut du clé en main
Julie — nutritionniste partenaire	B2B Pro	40	Consulter ses patients	Travail professionnel, rigueur

2.2 Adaptations par profil

Léa (Freemium) : - Wizard d'inscription guidé (6 étapes courtes) - Bandeau "Mode démonstration" quand données fictives - PremiumGuard avec blur (elle voit ce qu'elle peut débloquer) - Landing page avec prix lisibles (0€ clairement affiché)

Thomas (Premium) : - Dashboard synthétique (4 KPIs en haut) - Génération rapide de plans (1 clic) - Raccourcis clavier pour naviguer - Cache intelligent = pages réactives

Sarah (Premium+) : - Onglets Analytics avancés - Page Settings complète (profil, objectifs, abonnement) - Accès aux graphiques détaillés (macros) - Upgrade path clair (via page offres)

Marc (B2B) : - Page Clients dédiée (`/dashboard/clients`) - Theming dynamique (marque blanche possible) - Données anonymisées (pas d'exposition RGPD)

Julie (B2B Pro) : - Accès aux profils santé des patients - Documentation OpenAPI pour intégration - Export possible des données

---

3. Parcours d'adoption

3.1 Découverte (Awareness)

Landing page `/` : - Hero clair : "Votre parcours santé personnalisé" - 3 offres lisibles côte à côte (Freemium / Premium / Premium+) - CTAs multiples (haut de page, cards, bas de page) - Aucune friction : pas de popup, pas de demande email pour découvrir

3.2 Inscription (Consideration)

Wizard `/inscription` : - **6 étapes** plutôt qu'un formulaire long (cognitive load réduit) - **Progression visible** (barre + "Étape X sur 6") - **Persistance localStorage** → l'utilisateur peut fermer et revenir - **Validations contextuelles** (messages sous chaque champ) - **Navigation libre** (bouton Précédent toujours accessible)

3.3 Onboarding (Activation)

Première connexion → Dashboard : - **Données demo par défaut** si aucune donnée utilisateur - **Bandeau explicite** : "Uploadez un repas pour voir vos vraies stats" - **Sidebar avec tous les modules** (pas de cachette) - **Badges** 🔒 **/** 👑 sur les fonctionnalités premium (clarté sur ce qui est accessible)

3.4 Fidélisation (Retention)

- **Historique nutrition** : valorise les efforts
- **Graphiques progression** : poids, calories dans le temps
- **BMI card** : feedback santé immédiat
- **Plans repas/workout** : relance l'usage régulier

3.5 Upgrade (Conversion)

- **PremiumGuard** : l'utilisateur voit ce qu'il débloquerait (blur + preview)
- **UpgradeDialog** : modal avec avantages clairs
- **Page Settings > Abonnement** : CTA direct

---

4. Stratégies d'accessibilité

4.1 Accessibilité technique

Voir `04-Accessibilite-WCAG-RGAA.md` pour le détail WCAG 2.1 AA.

4.2 Accessibilité cognitive

- **Langage simple** : pas de jargon technique
- **Phrases courtes** : descriptions sous les champs limitées à 1 ligne
- **Icônes + texte** : toujours les deux (ex: "Se connecter" avec icône flèche)
- **Couleurs sémantiques** : vert = succès, rouge = erreur, jaune = alerte
- **Feedback immédiat** : toasts, loaders, transitions

4.3 Accessibilité situationnelle

- **Connexion faible** : `cachedFetch()` évite les refetchs inutiles
- **Backend down** : fallback demo + bandeau jaune (pas d'écran blanc)
- **Device mobile** : layout adapté (375px-1920px testé)
- **Clavier uniquement** : 100% navigable (testé Playwright + manuel)
- **Mode sombre** : `prefers-color-scheme` respecté

---

5. Formation & documentation

5.1 Pour les utilisateurs finaux

Le frontend **s'auto-explique** par le design : - Labels explicites - Placeholders montrant des exemples - Tooltips sur les icônes (via `aria-label`) - Messages d'erreur actionnables ("Entrez un nombre entre 18 et 100")

Pas besoin de formation : la plateforme vise le grand public (Millennials/Gen Z habitués aux UX modernes).

5.2 Pour les équipes internes

- **README** : instructions d'installation et démarrage
- **docs/** : documentation technique (11 fichiers fonctionnalités + TODO)
- **Storybook** : non implémenté (reporté post-MVP)
- **Types TypeScript** : autocomplétion IDE complète

5.3 Pour les partenaires B2B

- **Swagger UI** sur `/ui` (backend) : documentation API interactive
- **OpenAPI spec** sur `/doc` : import possible dans Postman/Insomnia
- **Types exportables** : possibilité de générer un SDK client

---

6. Gestion des erreurs & continuité de service

6.1 Principes

L'utilisateur ne doit **jamais** voir : - Un écran blanc - Une stack trace technique - Un message HTTP brut (404, 500) - Un état bloqué sans action possible

6.2 Mise en œuvre

API indisponible → Fallback demo + bandeau "Mode démonstration"

```
try {
  const data = await cachedFetch('/api/home');
  setData(data);
} catch {
  setData(DEMO_DATA);
  setIsDemo(true);
}
```

401 Non authentifié → Redirect auto vers `/connexion`

403 Premium requis → Toast explicite + `UpgradeDialog`

429 Too Many Requests → Toast "Trop de requêtes, réessayez dans X secondes"

Validation Zod → Message inline sous le champ, annoncé aux lecteurs d'écran

Upload échoué → Toast avec détail du problème (image trop lourde, format non supporté, IA indisponible)

---

7. Monitoring & feedback

7.1 Collecte de feedback

Dans l'application : - Mécanisme de correction des prédictions IA (`FeedbackRequest`) → améliore le modèle - Bouton "Signaler un problème" (roadmap)

Hors application : - GitHub Issues pour les bugs techniques - Enquête utilisateur trimestrielle (post-MVP)

7.2 Métriques suivies

Performance : - Lighthouse : Performance, Accessibilité, SEO, Best Practices - Core Web Vitals : LCP, FID, CLS

Qualité : - Taux de tests passants (objectif : 100%) - Couverture a11y (objectif : 0 violation critical/serious) - Lint ESLint (objectif : 0 warning)

Business (post-MVP) : - Taux de conversion Freemium → Premium - Rétention J+7 / J+30 - NPS (Net Promoter Score)

8. Gestion du changement technique

8.1 Mises à jour

- **Dépendances** : renouvelées en batch mensuel (Dependabot)
- **Next.js** : montée de version testée sur branche dédiée
- **React 19** : déjà adopté (dernière stable)
- **Breaking changes** : communiqués via CHANGELOG.md

8.2 Feature flags

Système de feature flags implicite via : - `PremiumGuard` (accès selon tier) - Variables d'environnement (`NEXT_PUBLIC_*`) - A/B testing possible via Vercel Edge Config (roadmap)

8.3 Rollback

- Déploiement sur Vercel : **rollback en 1 clic** vers version précédente
- Branches Git : chaque feature sur sa branche, merge sur main
- Tests CI : bloquent le deploy si violations a11y ou tests cassés

---

9. Roadmap d'amélioration

9.1 Court terme (Q2 2026)

- **i18n** : support anglais (viser les marchés UK/US)
- **Notifications push** (PWA + Service Worker)
- **Mode hors-ligne** pour le journal nutrition
- **Storybook** pour les composants UI

9.2 Moyen terme (Q3-Q4 2026)

- **App mobile iOS/Android** (React Native ou Expo)
- **Intégration objets connectés** (Web Bluetooth)

- **Consultations vidéo** (WebRTC)
- **Assistant vocal** (Web Speech API)

9.3 Long terme (2027+)

- **IA conversationnelle** (chat avec coach virtuel)
- **Communauté** (forum, challenges)
- **Gamification** (badges, streaks)

---

10. Conclusion

Le frontend HealthNext a été conçu avec une **approche centrée utilisateur** :

1. **Accessibilité systémique** : WCAG AA by default via Radix UI + tests axe-core automatisés
2. **Résilience** : fallbacks partout, aucun écran blanc possible
3. **Progressive disclosure** : Freemium → Premium → Premium+ avec guards clairs
4. **Multi-tenant B2B** : theming dynamique pour marque blanche
5. **Qualité** : 33 tests e2e passants, lint clean, build optimisé

Cette approche garantit une **adoption fluide** par tous les profils cibles, depuis l'étudiante curieuse jusqu'au gérant de salle de sport, tout en respectant les standards d'accessibilité les plus exigeants du marché français (RGAA).

Rapport de tests et couverture — Frontend HealthNext

Synthèse

La suite de tests end-to-end du frontend HealthNext est composée de **57 tests automatisés** exécutés via Playwright. Tous ces tests passent en environ 35 secondes sur une machine standard.

Résultats actuels : - Tests totaux : **57** - Tests passés : **57** (100%) - Tests échoués : **0** - Durée d'exécution : **~35 secondes** - Navigateurs couverts : Chromium (Firefox et WebKit désactivés en CI pour optimiser le temps de build)

Méthodologie de test

On a fait le choix d'une couverture principalement **end-to-end** (Playwright) plutôt que des tests unitaires (Jest/Vitest). Ce choix se justifie par :

- La nature de l'application (orientée parcours utilisateur)
- Le fait que les composants UI utilisés viennent de Shadcn/Radix, déjà testés en amont par leurs auteurs
- Playwright teste ce qui compte vraiment : le comportement réel dans un navigateur

Pour les flows qui dépendent du backend (auth, dashboard authentifié, appels API), on utilise `page.route()` de Playwright pour **mocker les réponses HTTP**. Cela permet de tester toute la logique frontend sans nécessiter le backend running en CI.

Détail des suites de tests

1. Parcours utilisateur critique (`user-journey.spec.ts`) — 5 tests

Les tests fondamentaux du parcours d'inscription et d'authentification.

#	Test	Description
1	Landing → Wizard	Clic sur le CTA "Commencer" redirige vers `/inscription`
2	Flow wizard complet	Renseignement âge, poids, taille, passage à l'étape objectifs
3	Validation âge	Un âge < 18 affiche un message d'erreur
4	Redirection auth	Accès à `/dashboard` sans session redirige vers `/connexion`
5	Formulaire login	Présence des champs email, mot de passe, bouton

2. Couverture des pages (`pages.spec.ts`) — 22 tests

Test page par page pour vérifier que chaque route répond correctement.

Landing Page (4 tests) - Hero visible avec CTA - Affichage des 3 offres (Freemium, Premium, Premium+) - Liens de navigation (connexion, inscription) - Footer avec copyright

Wizard d'inscription (4 tests) - Complétion des 3 premières étapes - Rejet d'un âge invalide - Rejet si taille manquante - Navigation arrière (bouton Précédent)

Page Connexion (2 tests) - Formulaire complet (email, password, submit) - Lien vers création de compte

Protection d'authentification (8 tests) - Redirection vers `/connexion` pour 8 routes protégées : - `/dashboard` - `/dashboard/nutrition` - `/dashboard/analytics` - `/dashboard/settings` - `/dashboard/workouts` - `/dashboard/clients` - `/dashboard/nutrition/history` - `/dashboard/nutrition/meal-plan`

Design responsive (4 tests) - Landing en viewport mobile (375×812) - Landing en viewport tablette (768×1024) - Inscription en viewport mobile - Connexion en viewport mobile

3. Accessibilité WCAG AA (`accessibility.spec.ts`) — 6 tests

Exécution d'axe-core sur 6 pages pour détecter les violations WCAG 2.1 AA.

Page	URL	Violations critiques
Landing	`/`	**0**
Inscription Wizard	`/inscription`	**0**
Dashboard	`/dashboard`	**0**
Analytics	`/dashboard/analytics`	**0**
Nutrition Tracker	`/dashboard/nutrition`	**0**
Settings	`/dashboard/settings`	**0**

Tags WCAG vérifiés : `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`.

Un rapport HTML détaillé est généré automatiquement dans `a11y-report/` via la commande `npm run test:a11y:report`.

4. Flow d'authentification (`auth-flow.spec.ts`) — 10 tests

Tests avec **mocks API** pour simuler les réponses backend.

#	Test	Description
1	Formulaire de login affiché	Champs email, password, bouton submit
2	Appel API login	Vérifie que POST `/api/auth/sign-in/email` est bien déclenché
3	Login réussi	Mock 200 + cookie de session
4	Lien vers inscription	Redirection vers `/inscription`
5	Bouton visible après échec	Le bouton reste accessible après un 401
6	Wizard : étape 1 sur 6	Indicateur de progression
7	Persistance localStorage	L'âge saisi est stocké dans `wizard-store`
8	Refus âge < 18	Message d'erreur affiché
9	Refus taille manquante	Validation Zod bloque
10	Navigation arrière	Le bouton Précédent restaure les données

5. Dashboard authentifié avec mocks (`dashboard-mocked.spec.ts`) — 14 tests

Tests des pages authentifiées, session simulée via cookie injecté.

Dashboard (4 tests) - Affichage des 4 KPIs (poids, calories, protéines, activité) - Bandeau démo quand API en erreur - Rendu des graphiques avec données réelles - Présence de la carte IMC

Analytics (3 tests) - Page chargée avec les stats - 3 onglets présents (vue d'ensemble, nutrition, poids) - Navigation entre onglets fonctionnelle

Settings (2 tests) - Page paramètres chargée - Onglets profil, objectifs, abonnement

Premium Guard (2 tests) - Free user bloqué sur la génération workout (403) - Premium user reçoit le plan

Gestion d'erreurs API (3 tests) - 401 → redirection vers login - 500 → fallback demo + bandeau - Toast d'erreur sur échec de génération

Commandes disponibles

```
# Lancer tous les tests
npm run test

# Parcours critique uniquement
npm run test:e2e

# Accessibilité uniquement
npm run test:a11y

# Accessibilité + rapport HTML
npm run test:a11y:report

# Afficher le dernier rapport Playwright
npm run test:report
```

Intégration continue

Le pipeline GitHub Actions exécute tous les tests à chaque push et pull request :

1. Installation des dépendances (`npm ci`)
2. Build de production (`npm run build`)
3. Exécution des tests (`npx playwright test`)
4. Upload du rapport HTML en artifact (rétention 7 jours)
5. Upload du rapport axe-core a11y en artifact (rétention 30 jours)

Le build échoue automatiquement si : - Un test e2e échoue - Des violations WCAG critical/serious sont détectées - Le linter ESLint trouve une erreur

Couverture effective

Ce qui est couvert

- **Routes publiques** : toutes testées (landing, connexion, inscription)
- **Routes protégées** : redirection vérifiée pour chacune
- **Flows d'authentification** : login/logout avec mocks réalistes
- **Dashboard & charts** : affichage des données et fallback
- **Accessibilité** : 6 pages principales auditées
- **Responsive** : viewports mobile, tablette, desktop
- **Gestion d'erreurs** : 401, 403, 500 gérés
- **Premium Guard** : blocage et déblocage testés

Ce qui n'est pas couvert (volontairement)

- **Tests de charge** : hors scope frontend (à faire côté backend/k6)
- **Tests de sécurité** : couverts par des outils dédiés (npm audit, Snyk)
- **Tests cross-browser Firefox/Safari** : désactivés en CI pour gagner du temps, peuvent être activés en local

Ce qui pourrait être amélioré

- Tests unitaires des hooks custom (`usePremiumStatus`) en Vitest
- Tests de composants isolés avec Storybook + Chromatic (post-MVP)
- Mesures Lighthouse automatisées en CI (Core Web Vitals)

Rapport HTML interactif

Playwright génère un rapport HTML détaillé après chaque exécution, accessible via :

```
playwright-report/index.html
```

Ce rapport contient : - Liste de tous les tests avec statut (pass/fail/skip) - Durée d'exécution par test - Traces complètes (vidéo + screenshots + network) en cas d'échec - Filtrage par statut, fichier, tag

Pour le consulter : `npx playwright show-report`.

Annexe : configuration Playwright

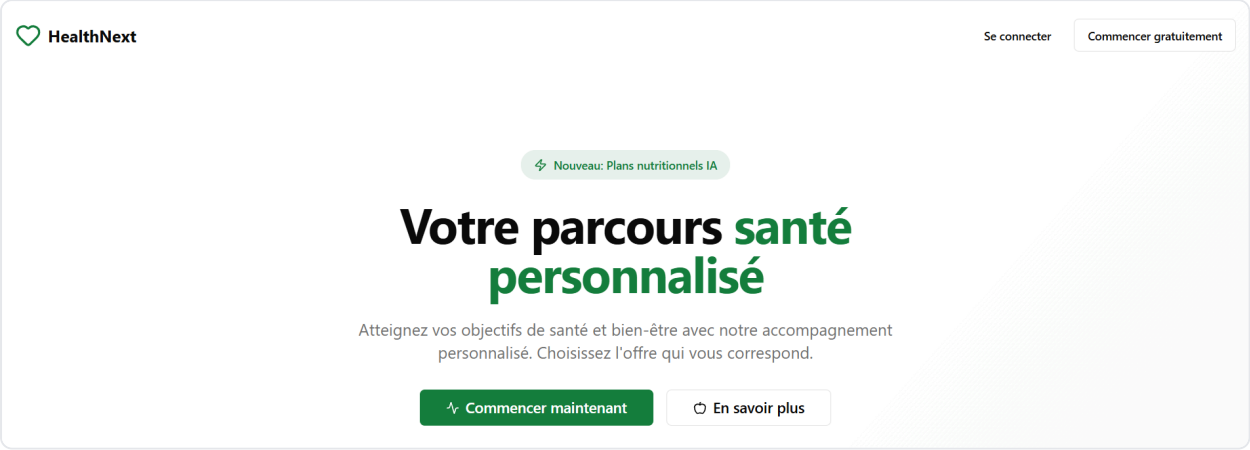
Extrait de `playwright.config.ts` :

```
export default defineConfig({
  testDir: "./e2e",
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: [
    ["html", { outputFolder: "playwright-report" }],
    ["list"],
  ],
  use: {
    baseURL: "http://localhost:3001",
    trace: "on-first-retry",
    video: "on-first-retry",
    screenshot: "only-on-failure",
  },
  webServer: {
    command: "npm run build && npx next start -p 3001",
    url: "http://localhost:3001",
    reuseExistingServer: !process.env.CI,
    timeout: 120000,
  },
});
```

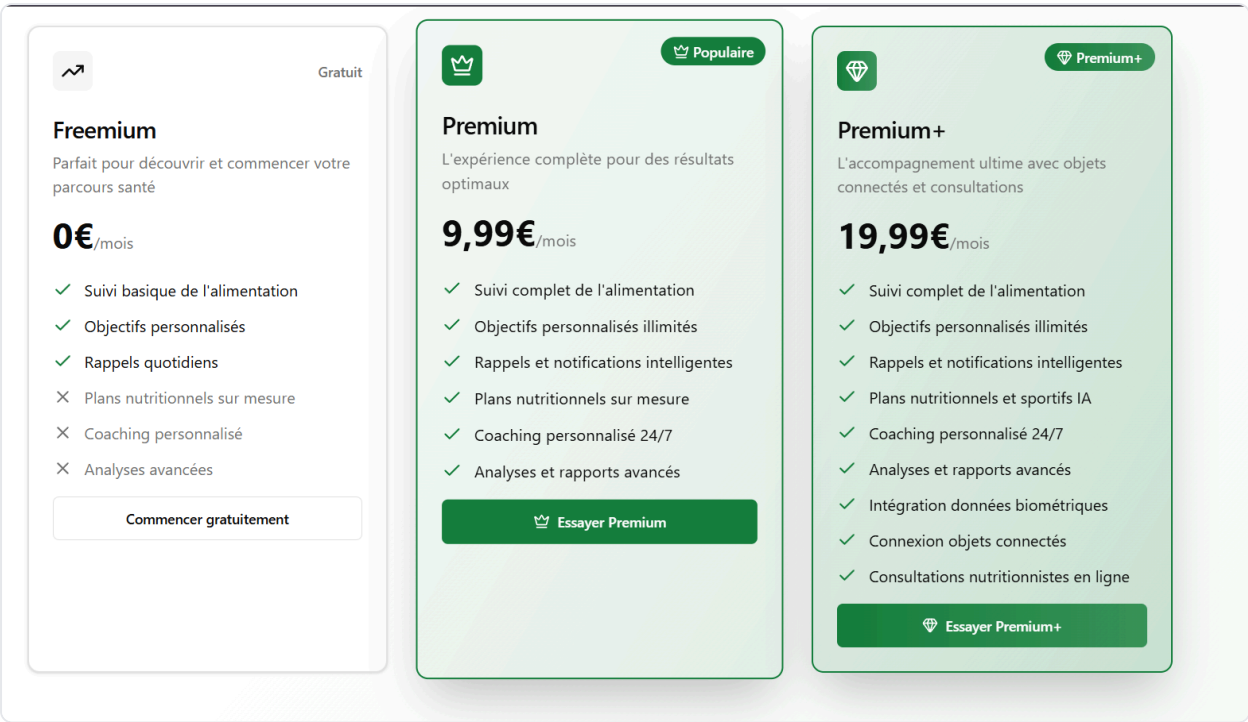
Le web server est démarré automatiquement au lancement des tests, avec réutilisation en local (gain de temps).

Annexe : Captures d'ecran

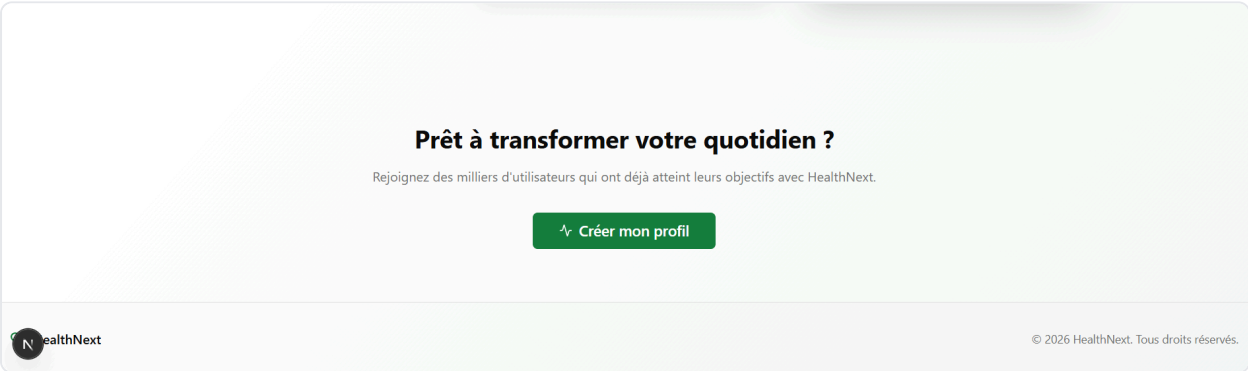
Landing Page



Hero de la landing page avec CTAs principaux

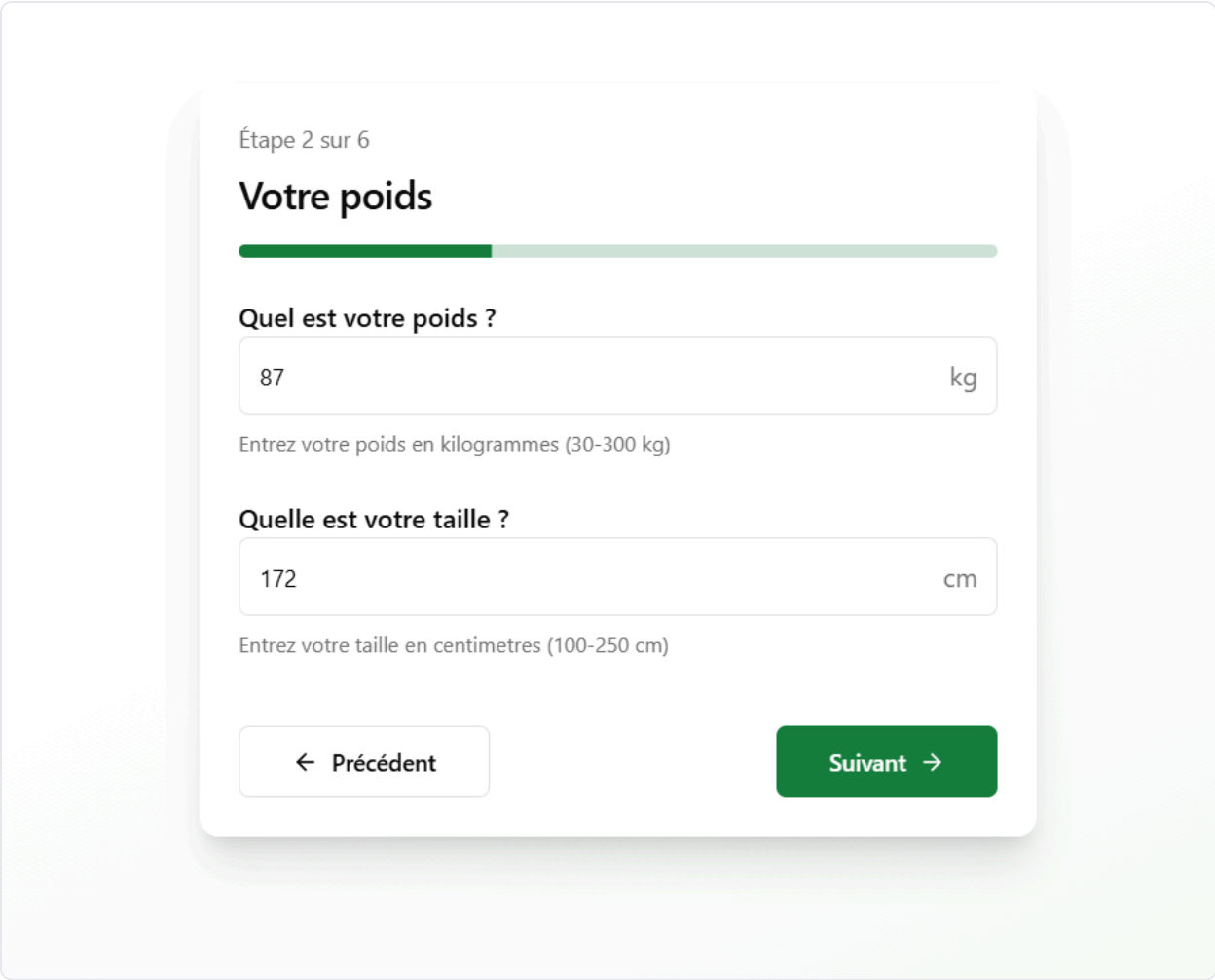


Section pricing : 3 offres Freemium / Premium / Premium+

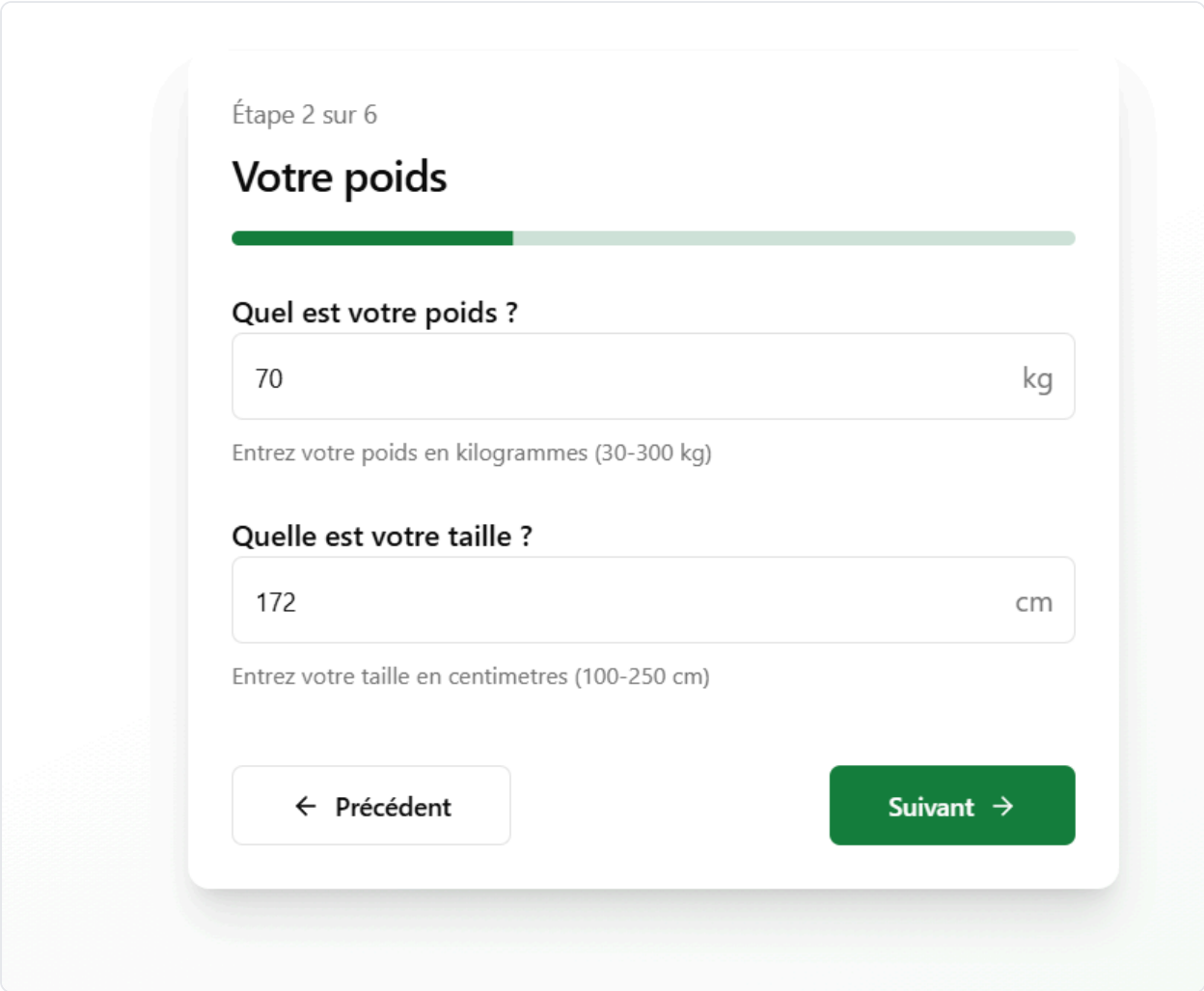


Section CTA final et footer

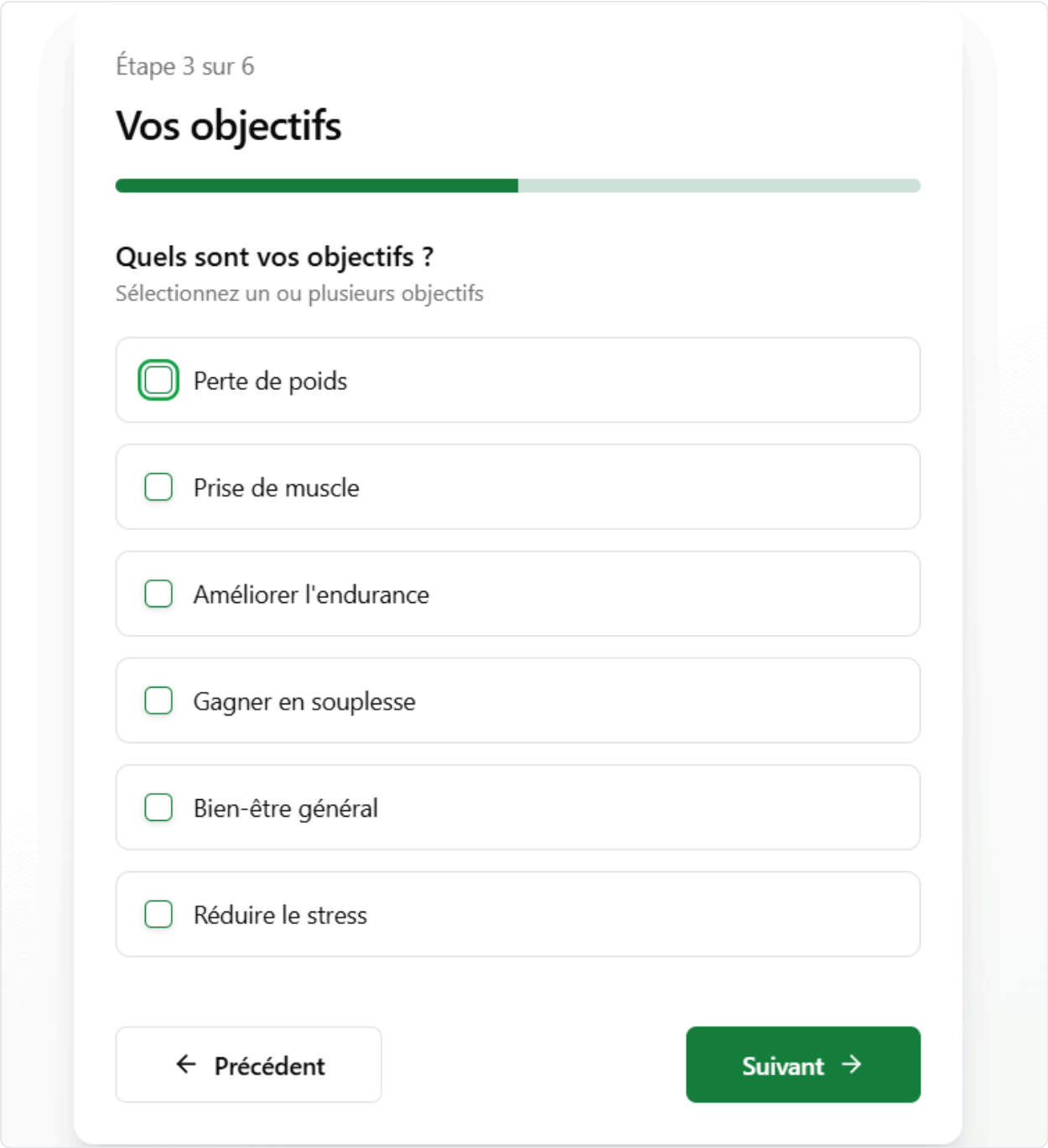
Inscription Wizard (6 etapes)



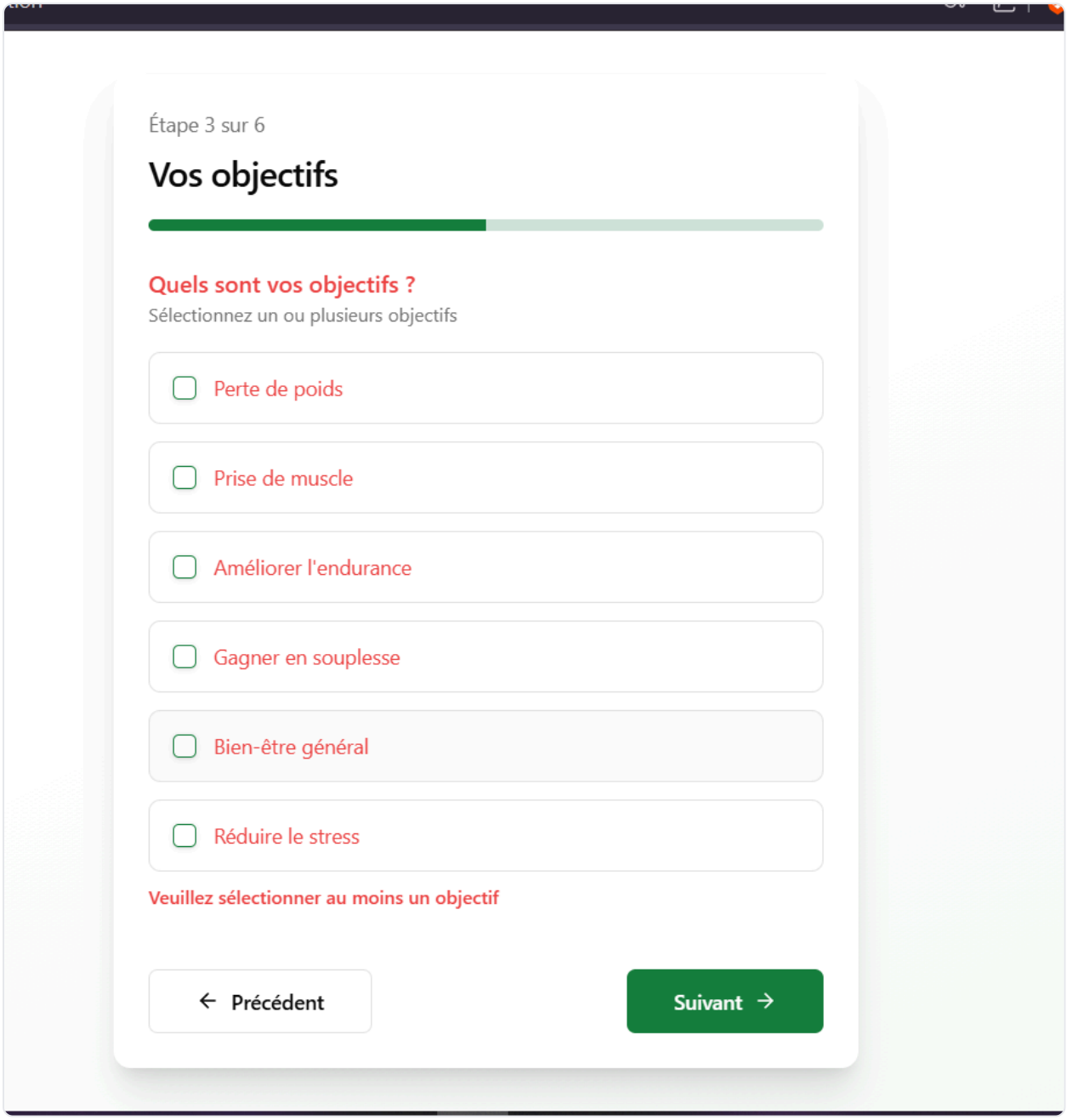
Etape 2 : poids et taille



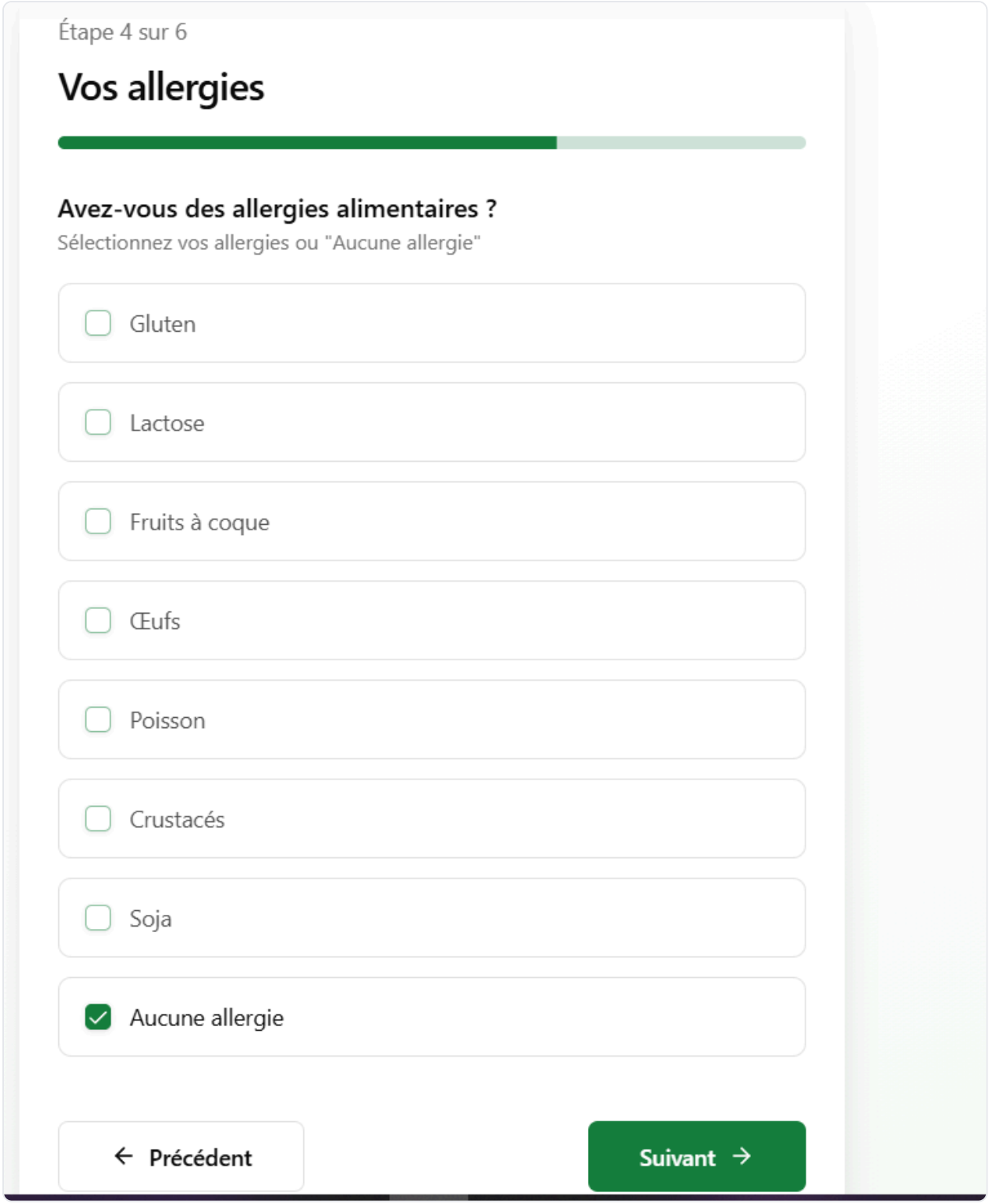
Etape 2 : validation des champs



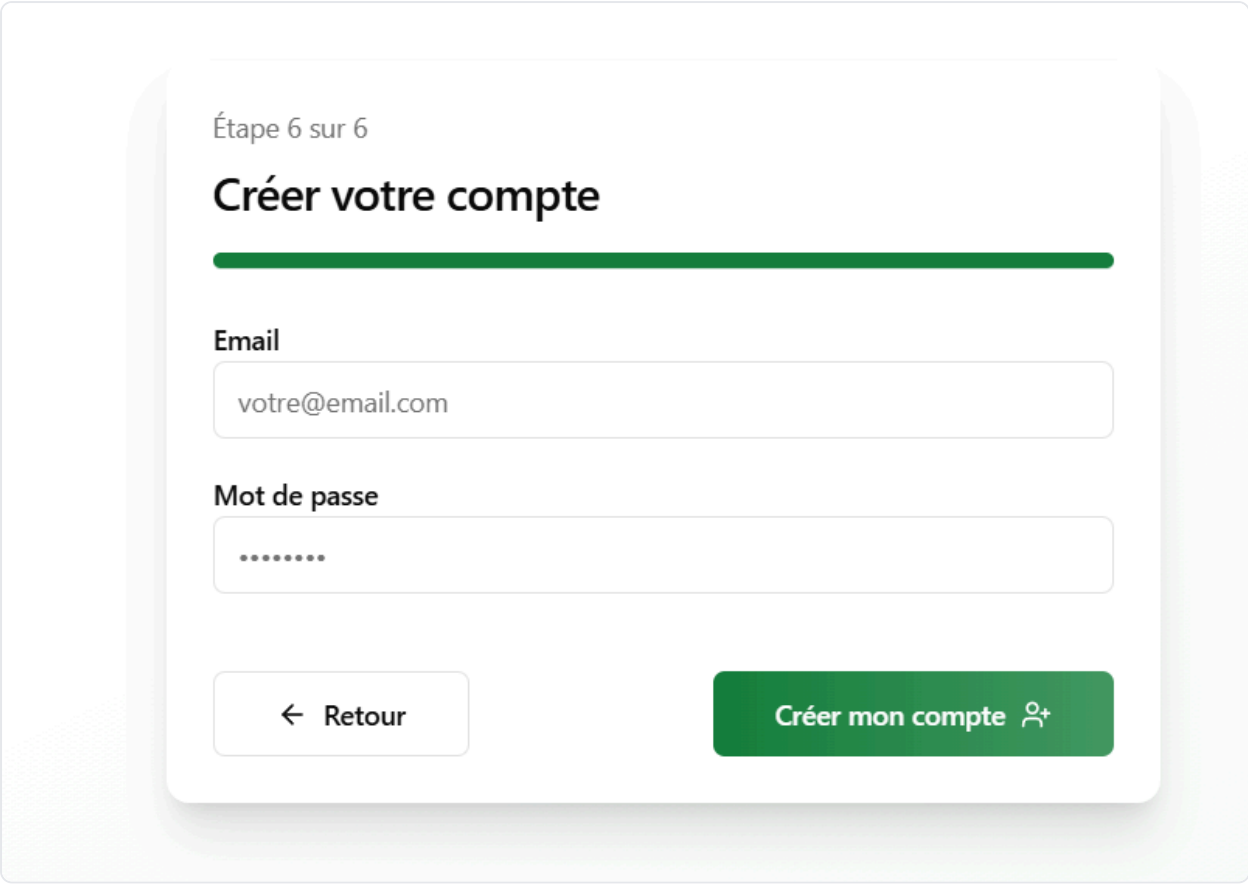
Etape 3 : selection d'objectifs



Etape 3 : message d'erreur si aucun objectif

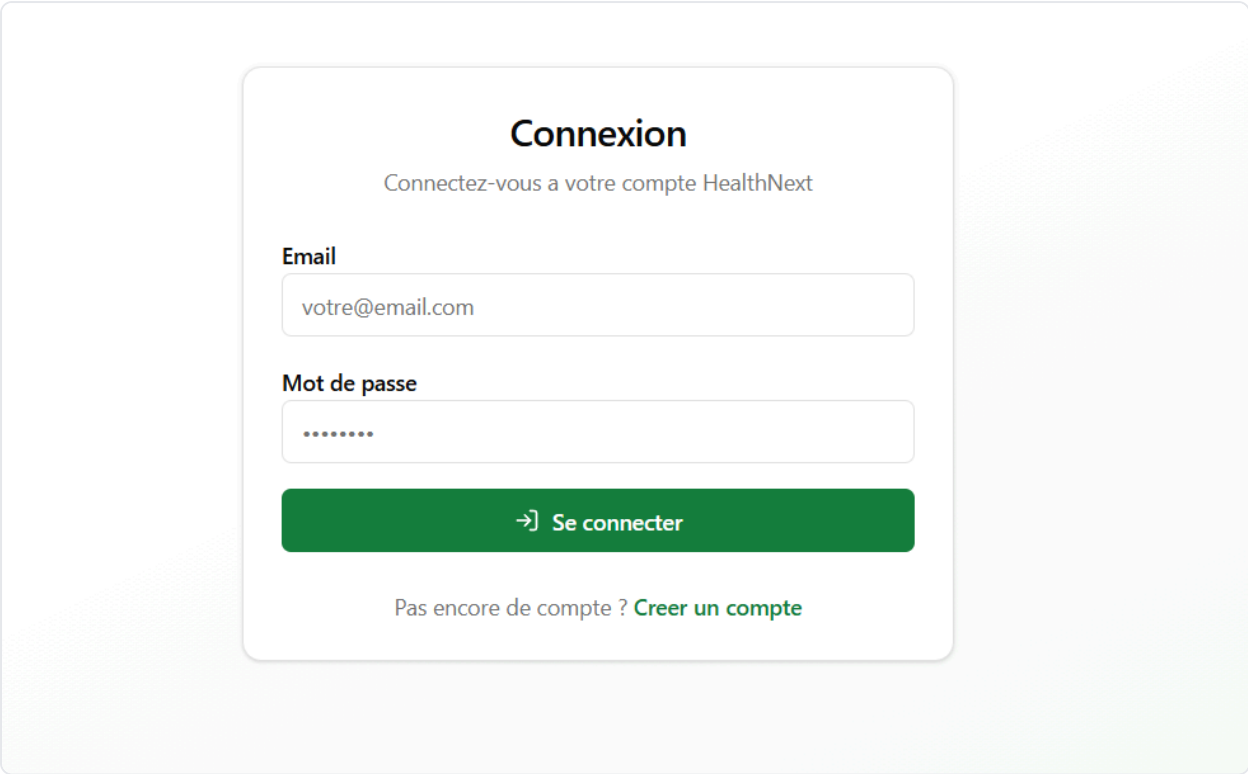


Etape 4 : allergies alimentaires



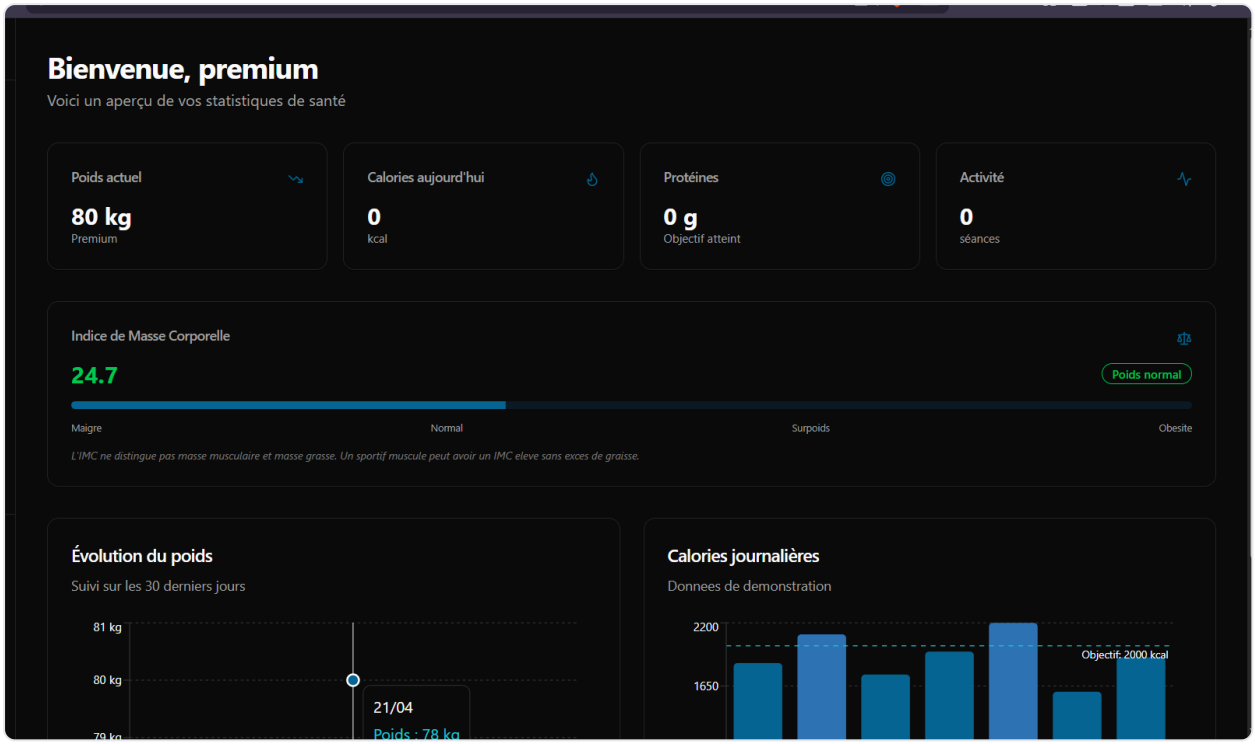
Etape 6 : creation du compte

Connexion



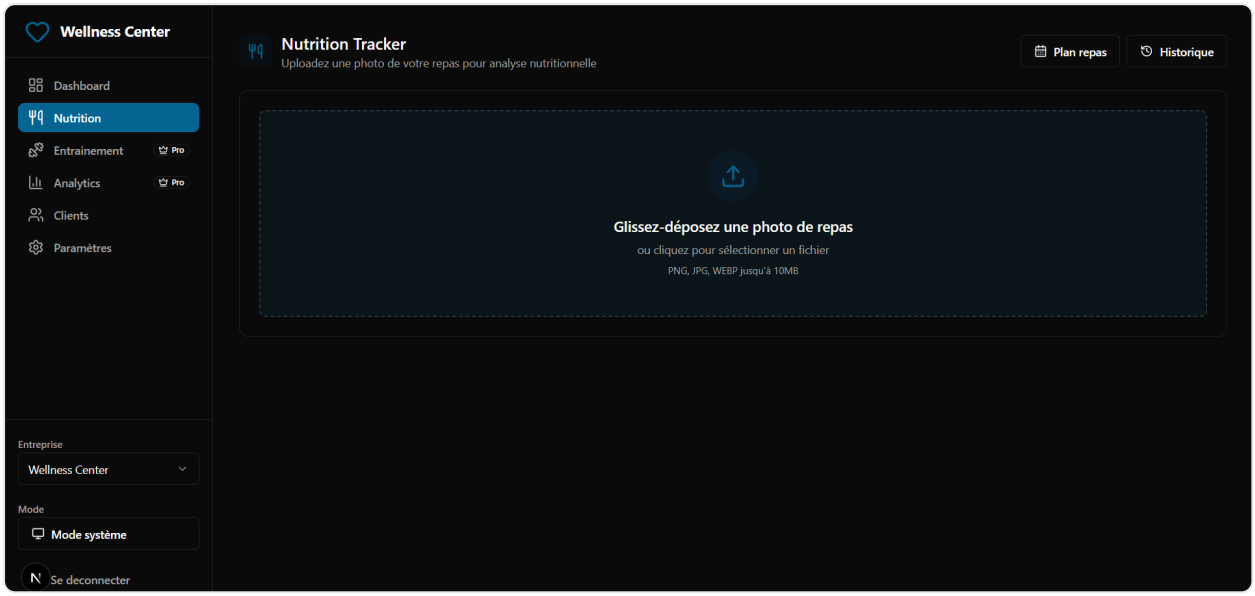
Page de connexion avec lien vers l'inscription

Dashboard



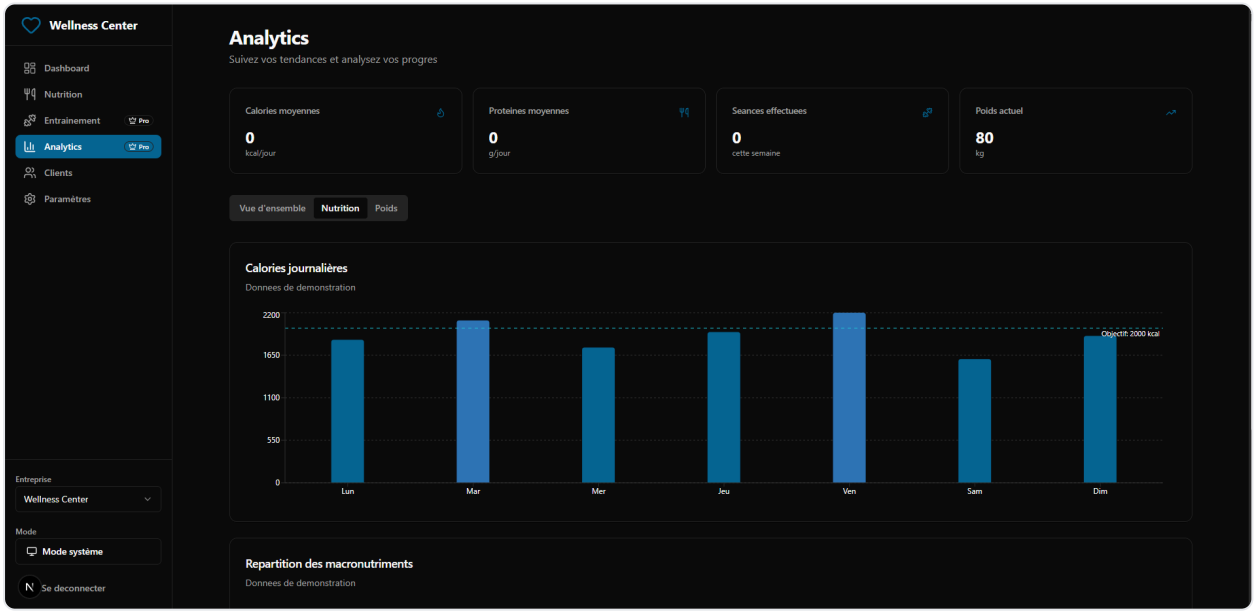
Dashboard principal : 4 KPIs, carte IMC, graphiques

Graphiques evolution poids et calories journalieres

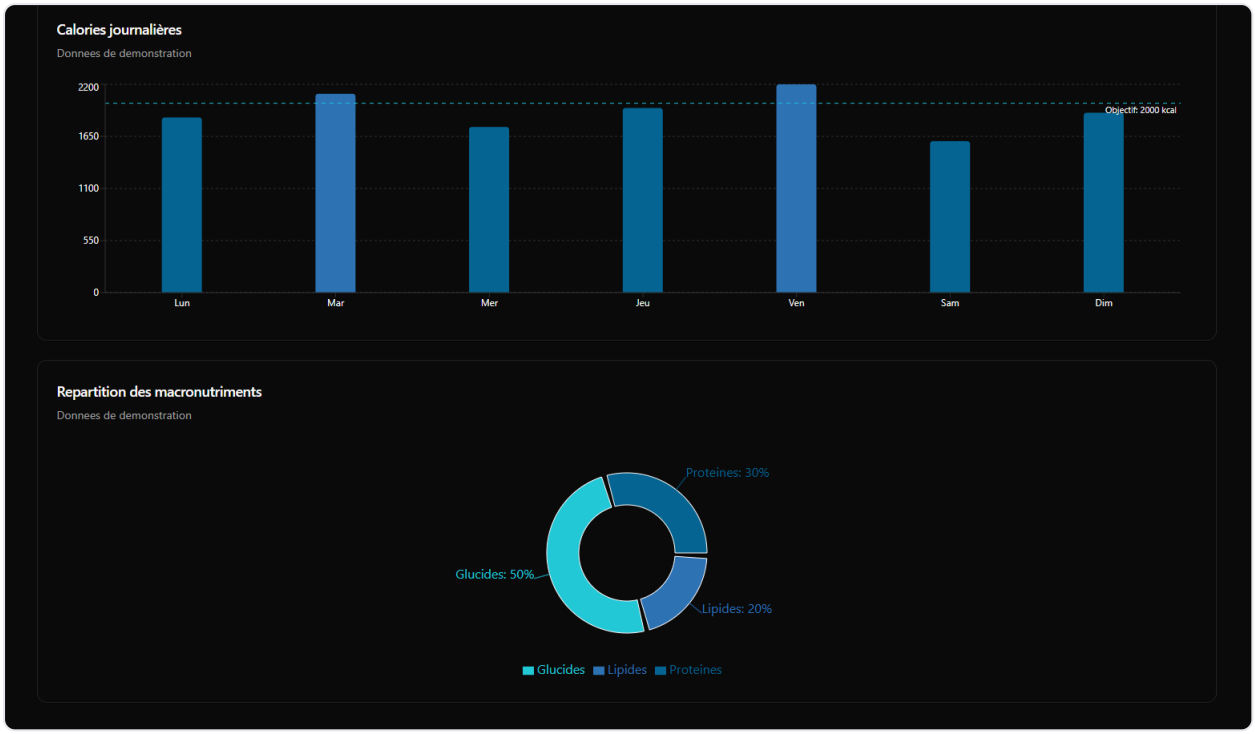


Zoom sur les graphiques (donnees reelles)

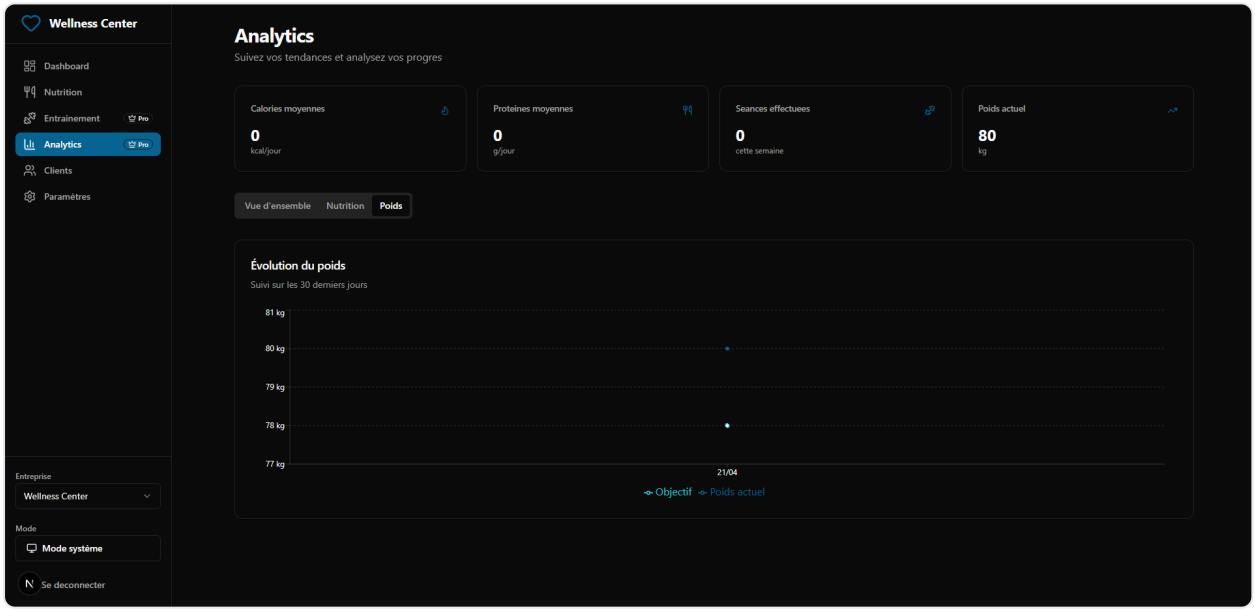
Analytics (Premium)



Onglet Vue d'ensemble : KPIs + graphique calories

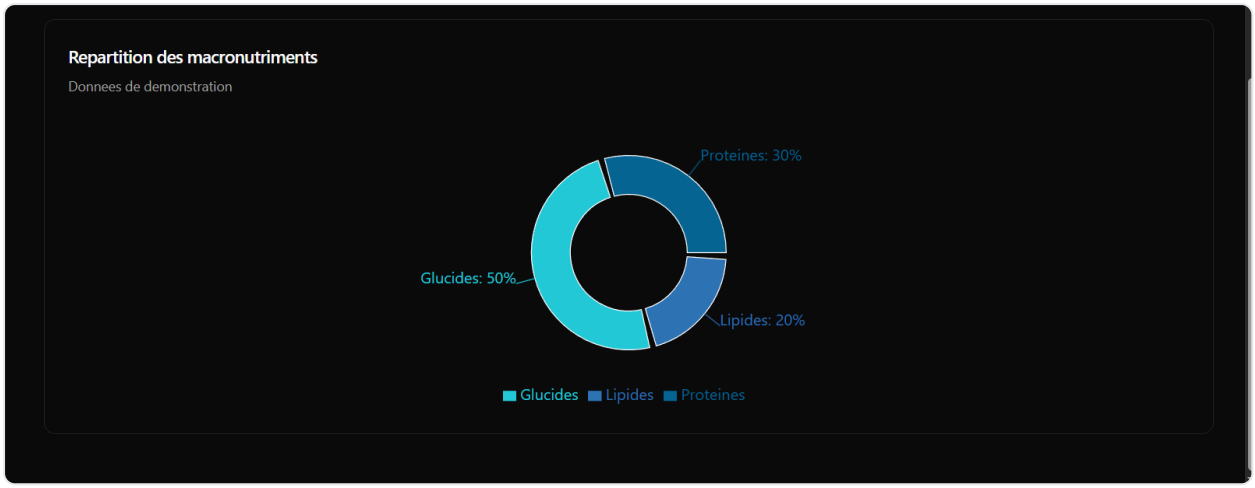


Graphique macros (PieChart) et calories

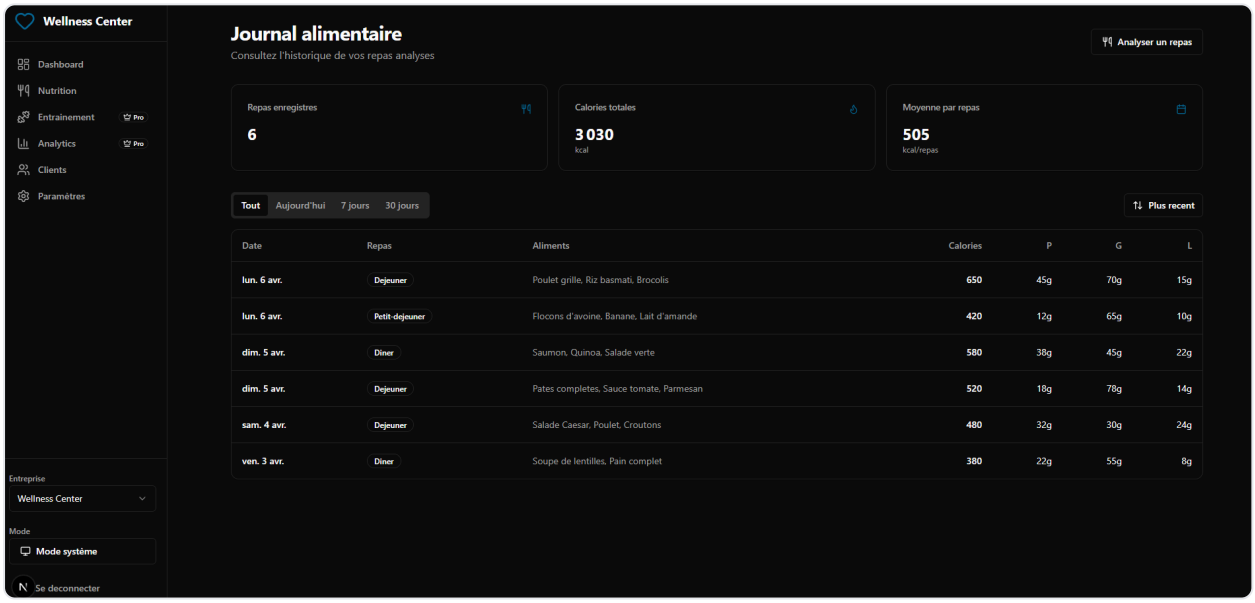


Onglet Poids : evolution 30 derniers jours

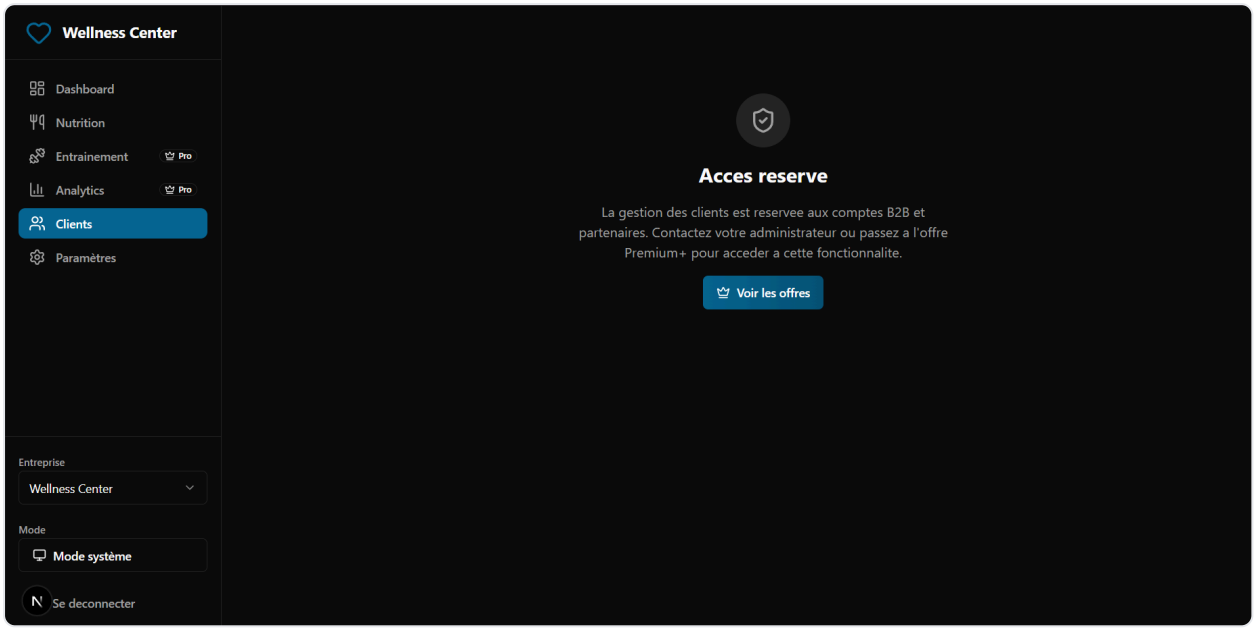
Nutrition



Upload d'une photo de repas (drag & drop)

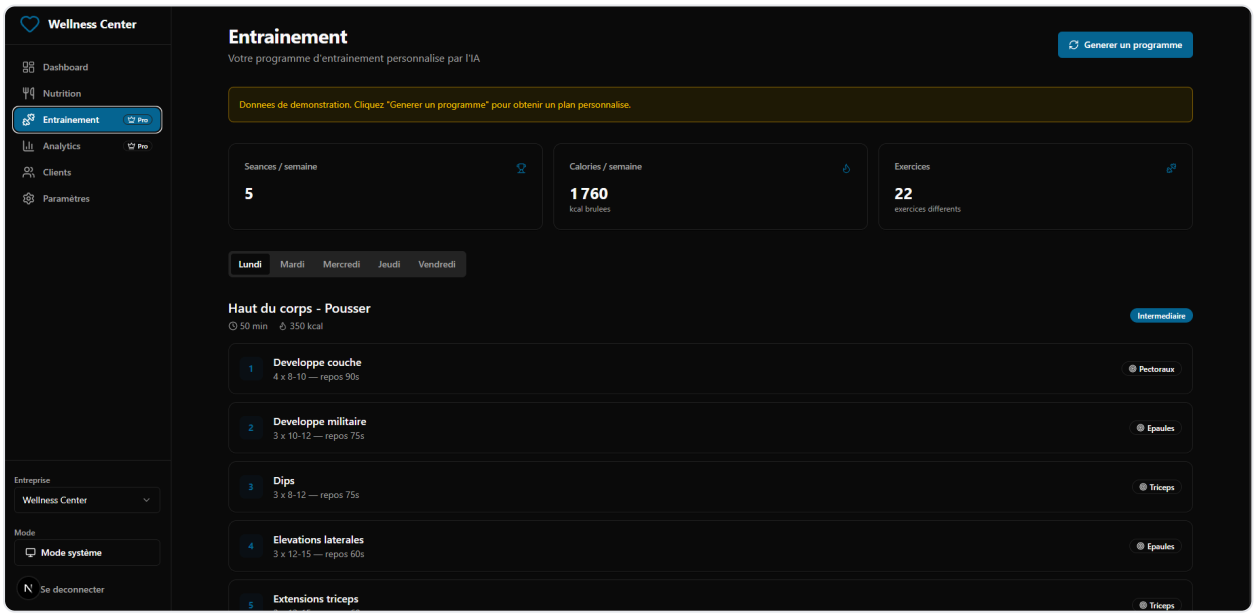


Journal alimentaire : historique avec filtres



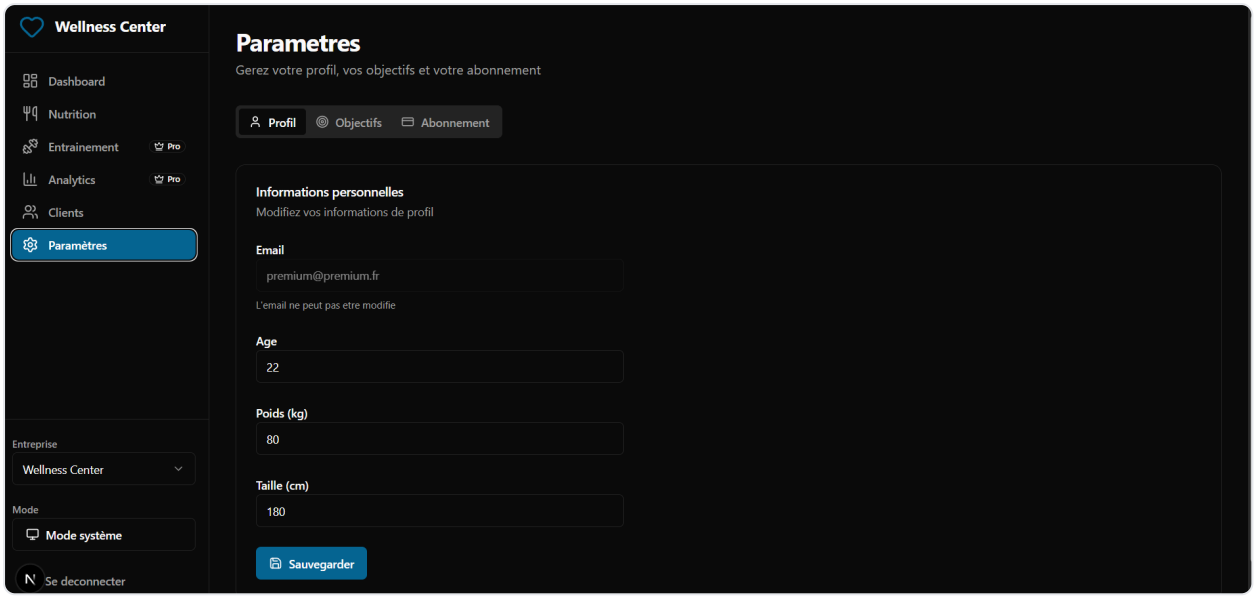
Plan de repas (Premium) genere par l'IA

Workouts (Premium)

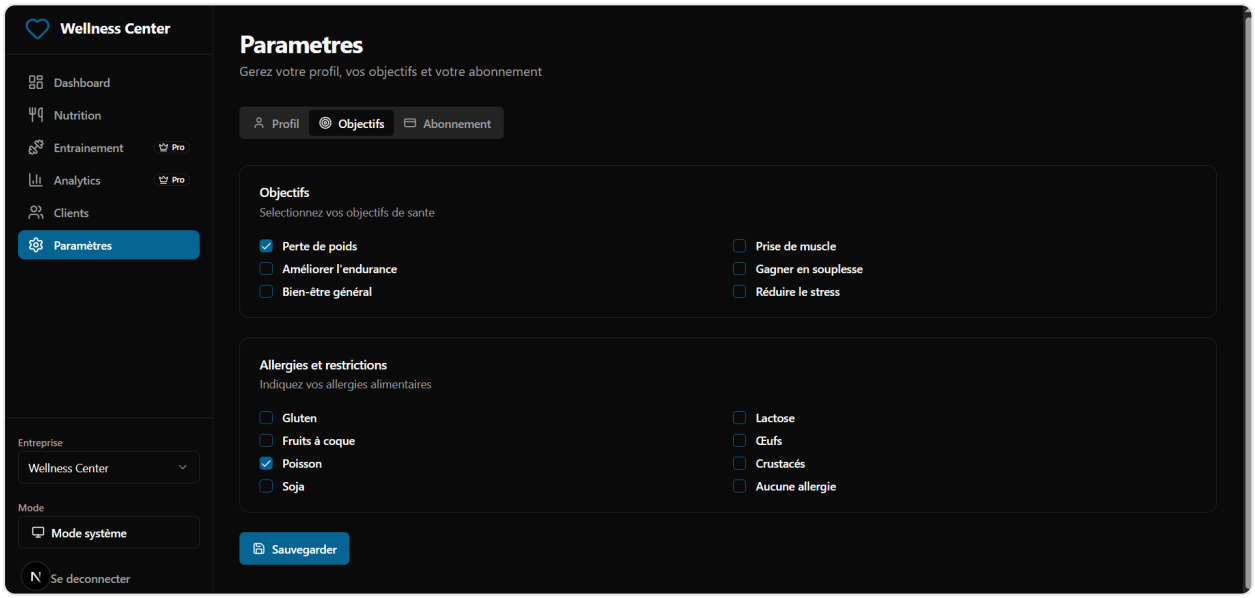


Programme d'entrainement hebdomadaire avec exercices

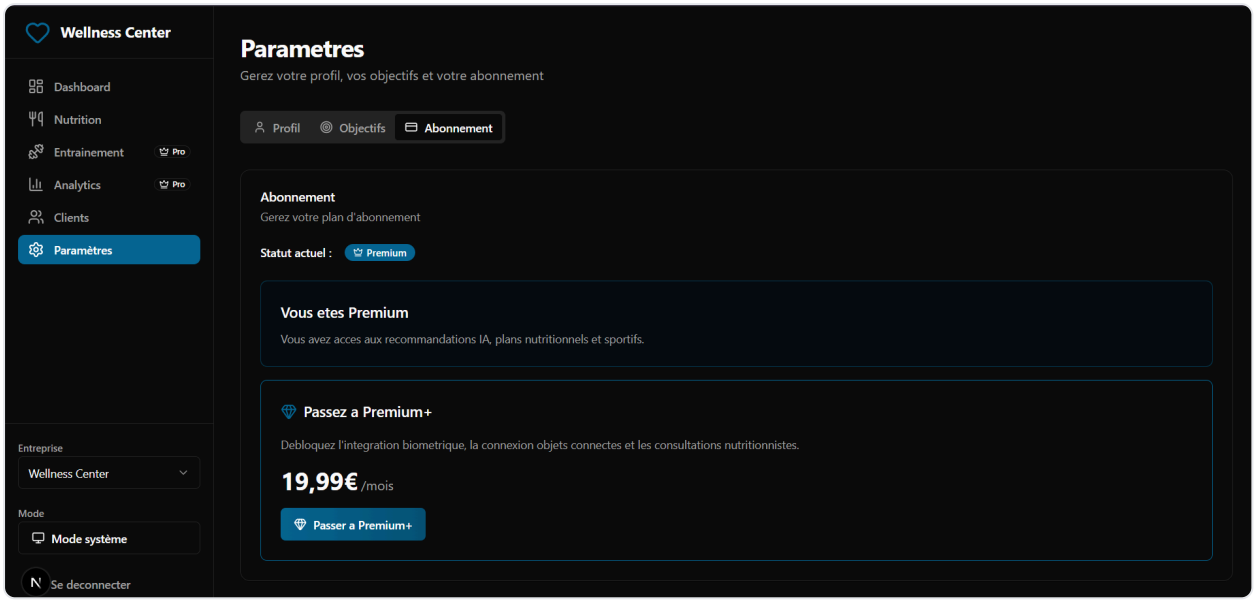
Settings



Onglet Profil : email, age, poids, taille

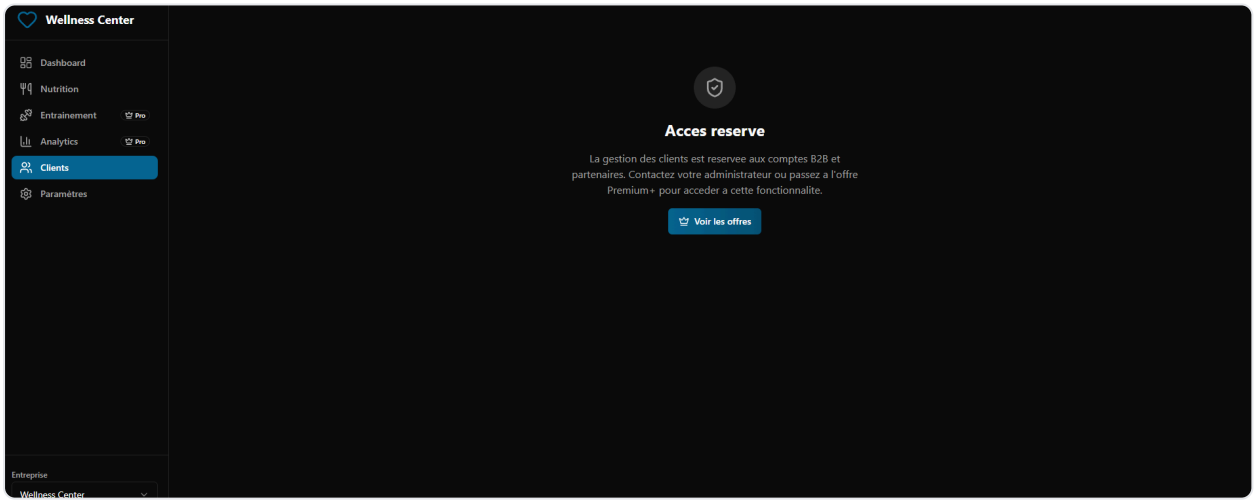


Onglet Objectifs et allergies alimentaires



Onglet Abonnement : badge Premium + upgrade Premium+

Clients B2B (Premium+)



Premium Guard : acces reserve aux comptes Premium+